

Neural Networks

EEE 443/543

Lecture 8 -
Neural Language Models

NEURAL LANGUAGE MODELS

Topics: natural language processing (NLP)

- Natural language processing is concerned with tasks involving language data
 - we will focus on text data NLP
 - speech processing is also NLP

https://info.usherbrooke.ca/hlarochelle/neural_networks/content.html

NEURAL LANGUAGE MODELS

Topics: natural language processing (NLP)

- Natural language processing is concerned with tasks involving language data
 - we will focus on text data NLP
 - speech processing is also NLP
- Neural networks adapted to the processing of text data
 - main issue: text data is inherently high dimensional

https://info.usherbrooke.ca/hlarochelle/neural_networks/content.html

Noam Chomsky

🌐 137 languages

Article [Talk](#)

[Read](#) [View source](#) [View history](#) [Tools](#)

From Wikipedia, the free encyclopedia



"Chomsky" redirects here. For other uses, see [Chomsky \(disambiguation\)](#).

Avram Noam Chomsky^[a] (born December 7, 1928) is an American professor and [public intellectual](#) known for his work in [linguistics](#), political activism, and [social criticism](#). Sometimes called "the father of modern linguistics",^[b] Chomsky is also a major figure in [analytic philosophy](#) and one of the founders of the field of [cognitive science](#). He is a laureate professor of linguistics at the [University of Arizona](#) and an [institute professor](#) emeritus at the [Massachusetts Institute of Technology](#) (MIT). Among the most cited living authors, Chomsky has written more than 150 books on topics such as linguistics, war, and politics. Ideologically, he aligns with [anarcho-syndicalism](#) and [libertarian socialism](#).

Born to [Ashkenazi Jewish](#) immigrants in [Philadelphia](#), Chomsky developed an early interest in [anarchism](#) from alternative bookstores in New York City. He studied at the [University of Pennsylvania](#). During his postgraduate work in the [Harvard Society of Fellows](#), Chomsky developed the theory of [transformational grammar](#) for which he earned his doctorate in 1955. That year he began teaching at MIT, and in 1957 emerged as a significant figure in linguistics with his landmark work *[Syntactic Structures](#)*, which played a major role in remodeling the study of language. From 1958 to 1959 Chomsky was a [National Science Foundation](#) fellow at the [Institute for Advanced Study](#). He created or co-created the [universal grammar](#) theory, the [generative grammar](#) theory, the [Chomsky hierarchy](#), and the [minimalist program](#). Chomsky also played a pivotal role in the decline of linguistic [behaviorism](#), and was particularly critical of the work of [B. F. Skinner](#).

Noam Chomsky



Chomsky in 2017

Born

Avram Noam Chomsky
December 7, 1928 (age 94)
[Philadelphia](#), Pennsylvania, U.S.

Linguistic theory

Main article: [Linguistics of Noam Chomsky](#)

https://en.wikipedia.org/wiki/Noam_Chomsky

The basis of Chomsky's linguistic theory lies in [biolinguistics](#), the linguistic school that holds that the principles underpinning the structure of language are biologically preset in the human mind and hence genetically inherited.^[146] He argues that all humans share the same underlying linguistic structure, irrespective of sociocultural differences.^[147] In adopting this position Chomsky rejects the [radical behaviorist](#) psychology of [B. F. Skinner](#), who viewed speech, thought, and all behavior as a completely learned product of the interactions between organisms and their environments.

OPINION
GUEST ESSAY

Noam Chomsky: The False Promise of ChatGPT

March 8, 2023

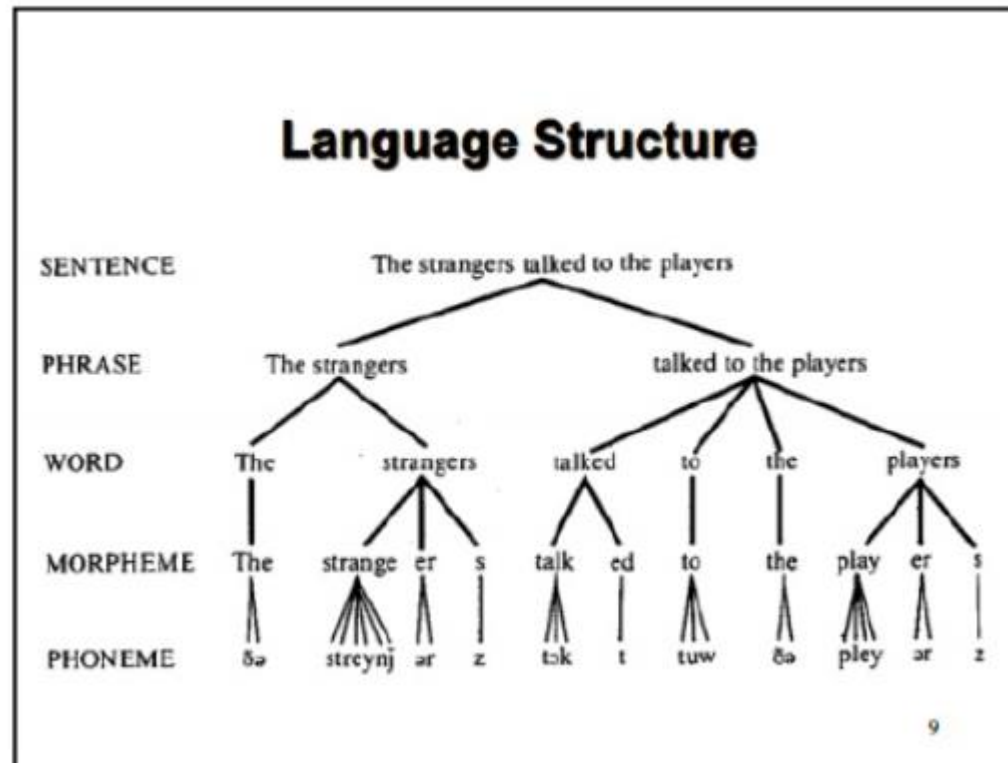
By Noam Chomsky, Ian Roberts and Jeffrey Watumull

Dr. Chomsky and Dr. Roberts are professors of linguistics. Dr. Watumull is a director of artificial intelligence at a science and technology company.

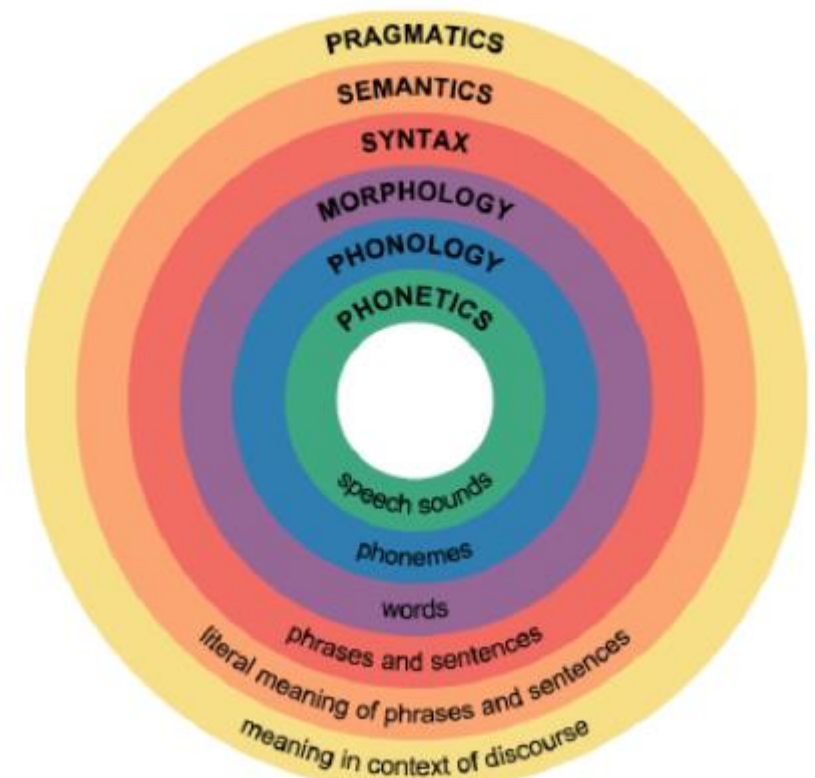
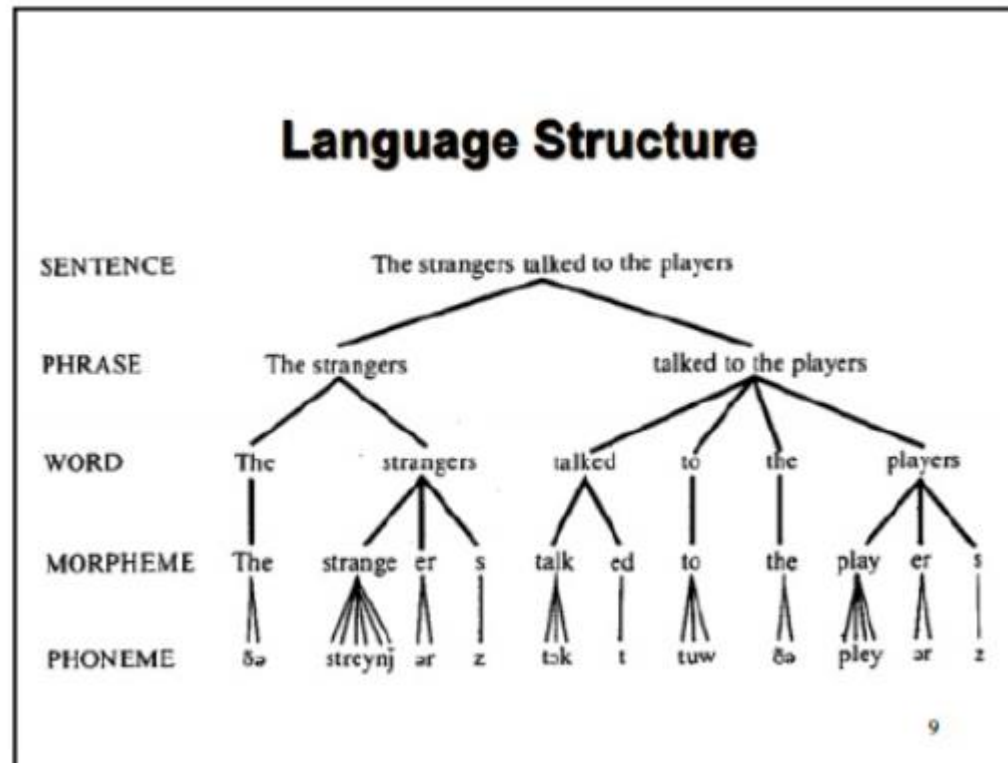
https://edisciplinas.usp.br/pluginfile.php/7614933/mod_resource/content/1/Opinion%20-%20Noam%20Chomsky%20The%20False%20Promise%20of%20ChatGPT%20-%20The%20New%20York%20Times.pdf

STRUCTURE OF LANGUAGE

STRUCTURE OF LANGUAGE



STRUCTURE OF LANGUAGE



LANGUAGE MODELING

LANGUAGE MODELING

Topics: language modeling

- A language model is a probabilistic model that assigns probabilities to any sequence of words

$$p(w_1, \dots, w_T)$$

LANGUAGE MODELING

Topics: language modeling

- A language model is a probabilistic model that assigns probabilities to any sequence of words

$$p(w_1, \dots, w_T)$$

- language modeling is the task of learning a language model that assigns high probabilities to well formed sentences
- plays a crucial role in speech recognition and machine translation systems

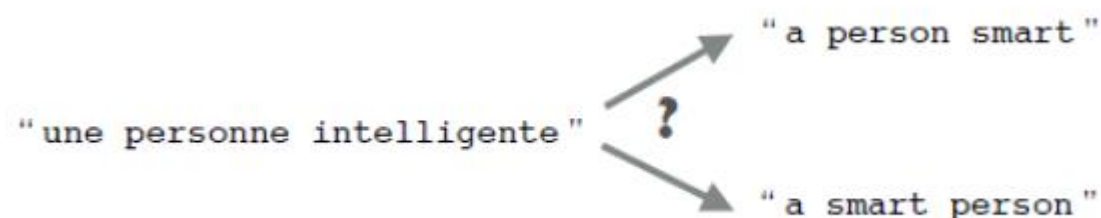
LANGUAGE MODELING

Topics: language modeling

- A language model is a probabilistic model that assigns probabilities to any sequence of words

$$p(w_1, \dots, w_T)$$

- language modeling is the task of learning a language model that assigns high probabilities to well formed sentences
- plays a crucial role in speech recognition and machine translation systems



TOKENIZATION

- Typical preprocessing steps of text data
 - tokenize text (from a long string to a list of token strings)

What is Tokenization?

Tokenization breaks text into smaller parts for easier machine analysis, helping machines understand human language.

Sep 2023 · 9 min read

Tokenization, in the realm of Natural Language Processing (NLP) and **machine learning**, refers to the process of converting a sequence of text into smaller parts, known as tokens. These tokens can be as small as characters or as long as words. The primary reason this process matters is that it helps machines understand human language by breaking it down into bite-sized pieces, which are easier to analyze.

<https://www.datacamp.com/blog/what-is-tokenization>

A string is constructed by piecing different characters together. Characters come together to build a word. Then many words come together to form a sentence, and sentences form a paragraph. Many paragraphs form a document, and so on.

*These units that build up the text corpus are **tokens** and the process of splitting a text sequence into its tokens is **tokenization**.*

A string is constructed by piecing different characters together. Characters come together to build a word. Then many words come together to form a sentence, and sentences form a paragraph. Many paragraphs form a document, and so on.

*These units that build up the text corpus are **tokens** and the process of splitting a text sequence into its tokens is **tokenization**.*

- characters
- words
- punctuations

A string is constructed by piecing different characters together. Characters come together to build a word. Then many words come together to form a sentence, and sentences form a paragraph. Many paragraphs form a document, and so on.

*These units that build up the text corpus are **tokens** and the process of splitting a text sequence into its tokens is **tokenization**.*

```
['Seek wealth, not money or status.',  
 'Wealth is having assets that earn while you sleep.',  
 'Money is how we transfer time and wealth.',  
 'Status is your place in the social hierarchy.']
```

- characters
- words
- punctuations

A string is constructed by piecing different characters together. Characters come together to build a word. Then many words come together to form a sentence, and sentences form a paragraph. Many paragraphs form a document, and so on.

*These units that build up the text corpus are **tokens** and the process of splitting a text sequence into its tokens is **tokenization**.*

- characters
- words
- punctuations

```
['Seek wealth, not money or status.',  
 'Wealth is having assets that earn while you sleep.',  
 'Money is how we transfer time and wealth.',  
 'Status is your place in the social hierarchy.']
```

RegexTokenizer

```
In [ ]: # import  
        from nltk.tokenize import RegexTokenizer  
  
        # word tokenization  
        word_tokenizer = RegexTokenizer("[\w']+")  
        print(word_tokenizer.tokenize(tweet))  
        print(f"# tokens: {len(word_tokenizer.tokenize(tweet))}")
```

```
['Seek', 'wealth', 'not', 'money', 'or', 'status', 'Wealth', 'is', 'having',  
 'assets', 'that', 'earn', 'while', 'you', 'sleep', 'Money', 'is', 'how', 'we',  
 'transfer', 'time', 'and', 'wealth', 'Status', 'is', 'your', 'place', 'in',  
 'the', 'social', 'hierarchy']
```

```
# tokens: 31
```

WordPunctTokenizer

```
In [ ]: # import  
        from nltk.tokenize import WordPunctTokenizer  
  
        # word tokenization  
        word_tokenizer = WordPunctTokenizer()  
        print(word_tokenizer.tokenize(tweet))  
        print(f"# tokens: {len(word_tokenizer.tokenize(tweet))}")
```

```
['Seek', 'wealth', ',', 'not', 'money', 'or', 'status', '.', 'Wealth', 'is',  
 'having', 'assets', 'that', 'earn', 'while', 'you', 'sleep', '.', 'Money', 'is',  
 'how', 'we', 'transfer', 'time', 'and', 'wealth', '.', 'Status', 'is', 'your',  
 'place', 'in', 'the', 'social', 'hierarchy', '.']
```

<https://medium.com/@utkarsh.kant/tokenization-a-complete-guide-3f2dd56c0682>

TOKENIZATION

- Typical preprocessing steps of text data
 - tokenize text (from a long string to a list of token strings)
 - for many datasets, this has already been done for you
 - splitting into tokens based on spaces and separating punctuation is good enough

TOKENIZATION

- Typical preprocessing steps of text data
 - tokenize text (from a long string to a list of token strings)
 - for many datasets, this has already been done for you
 - splitting into tokens based on spaces and separating punctuation is good enough

"He's spending 7 days in San
Francisco."



"He"
"'s"
"spending"
"7"
"days"
"in"
"San Francisco"
"."

LEMMATIZATION

Topics: lemmatization

- Typical preprocessing steps of text data

WHAT IS LEMMATIZATION?

Lemmatization is a text pre-processing technique used in natural language processing (NLP) models to break a word down to its root meaning to identify similarities. For example, a lemmatization algorithm would reduce the word *better* to its root word, or lemme, *good*.

<https://builtin.com/machine-learning/lemmatization>

WHAT IS LEMMATIZATION?

Lemmatization is a text pre-processing technique used in natural language processing (NLP) models to break a word down to its root meaning to identify similarities. For example, a lemmatization algorithm would reduce the word *better* to its root word, or lemme, *good*.

<https://builtin.com/machine-learning/lemmatization>

```
from nltk.stem import WordNetLemmatizer
nltk.download("wordnet")
nltk.download("omw-1.4")

# Initialize wordnet lemmatizer
wnl = WordNetLemmatizer()

# Example inflections to reduce
example_words = ["program", "programming", "programer", "programs", "programmed"]

# Perform lemmatization
print("{0:20}{1:20}".format("--Word--", "--Lemma--"))
for word in example_words:
    print("{0:20}{1:20}".format(word, wnl.lemmatize(word, pos="v")))

"""
--Word--          --Lemma--
program           program
programming       program
programer         programer
programs          program
programmed        program
"""
```

<https://www.datacamp.com/tutorial/stemming-lemmatization-python>

LEMMATIZATION

Topics: lemmatization

- Typical preprocessing steps of text data
 - the specific lemmatization will depend on the problem we want to solve
 - we can remove variations of words that are not relevant to the task at hand

LEMMATIZATION

Topics: lemmatization

- Typical preprocessing steps of text data
 - the specific lemmatization will depend on the problem we want to solve
 - we can remove variations of words that are not relevant to the task at hand

" He "
" 's "
" spending "
" 7 "
" days "
" in "
" San Francisco "
" . "



" he "
" be "
" spend "
" NUMBER "
" day "
" in "
" San Francisco "
" . "

VOCABULARY

Topics: vocabulary

- Typical preprocessing steps of text data

VOCABULARY

Topics: vocabulary

- Typical preprocessing steps of text data
 - form vocabulary of words that maps lemmatized words to a unique ID (position of word in vocabulary)

VOCABULARY

Topics: vocabulary

- Typical preprocessing steps of text data
 - form vocabulary of words that maps lemmatized words to a unique ID (position of word in vocabulary)
 - different criteria can be used to select which words are part of the vocabulary
 - pick most frequent words
 - ignore uninformative words from a user-defined short list (ex.: " the ", " a ", etc.)

VOCABULARY

Topics: vocabulary

- Typical preprocessing steps of text data
 - form vocabulary of words that maps lemmatized words to a unique ID (position of word in vocabulary)
 - different criteria can be used to select which words are part of the vocabulary
 - pick most frequent words
 - ignore uninformative words from a user-defined short list (ex.: "the", "a", etc.)
 - all words not in the vocabulary will be mapped to a special "out-of-vocabulary" ID

VOCABULARY

Topics: vocabulary

- Typical preprocessing steps of text data
 - form vocabulary of words that maps lemmatized words to a unique ID (position of word in vocabulary)
 - different criteria can be used to select which words are part of the vocabulary
 - pick most frequent words
 - ignore uninformative words from a user-defined short list (ex.: " the ", " a ", etc.)
 - all words not in the vocabulary will be mapped to a special "out-of-vocabulary" ID
- Typical vocabulary sizes will vary between 10 000 and 250 000

CATEGORICAL FEATURES

Topics: vocabulary

Vocabulary

Word	w
" the "	1
" and "	2
" dog "	3
" . "	4
" oov "	5

CATEGORICAL FEATURES

Topics: vocabulary

- Example:

" the "
" cat "
" and "
" the "
" dog "
" play "
" . "

Vocabulary

Word	w
" the "	1
" and "	2
" dog "	3
" . "	4
" oov "	5

CATEGORICAL FEATURES

Topics: vocabulary

- Example:

"the "
"cat "
"and "
"the "
"dog "
"play "
" . "

Vocabulary

Word	w
"the "	1
"and "	2
"dog "	3
" . "	4
" oov "	5



1
5
2
1
3
5
4

CATEGORICAL FEATURES

Topics: vocabulary

- Example:

"the "
"cat "
"and "
"the "
"dog "
"play "
" . "

Vocabulary

Word	w
"the "	1
"and "	2
"dog "	3
" . "	4
"oov "	5



1
5
2
1
3
5
4

- We will note word IDs with the symbol w
 - can think of w as a categorical feature for the original word
 - we will sometimes refer to w as a word, for simplicity

ONE-HOT ENCODING

Topics: one-hot encoding

- From its word ID, we get a basic representation of a word through the one-hot encoding of the ID
 - the one-hot vector of an ID is a vector filled with 0s, except for a 1 at the position associated with the ID
 - ex.: for vocabulary size $D=10$, the one-hot vector of word ID $w=4$ is

$$e(w) = [0\ 0\ 0\ 1\ 0\ 0\ 0\ 0\ 0\ 0]$$

ONE-HOT ENCODING

Topics: one-hot encoding

- From its word ID, we get a basic representation of a word through the one-hot encoding of the ID
 - the one-hot vector of an ID is a vector filled with 0s, except for a 1 at the position associated with the ID
 - ex.: for vocabulary size $D=10$, the one-hot vector of word ID $w=4$ is

$$e(w) = [0\ 0\ 0\ 1\ 0\ 0\ 0\ 0\ 0\ 0]$$

- a one-hot encoding makes no assumption about word similarity
 - $\|e(w) - e(w')\|^2 = 0$ if $w = w'$
 - $\|e(w) - e(w')\|^2 = 2$ if $w \neq w'$
 - all words are equally different from each other
- this is a natural representation to start with, though a poor one

ONE-HOT ENCODING MODEL

- The major problem with the one-hot representation is that it is very high-dimensional
 - the dimensionality of $e(w)$ is the size of the vocabulary
 - a typical vocabulary size is $\approx 100\,000$

ONE-HOT ENCODING MODEL

- The major problem with the one-hot representation is that it is very high-dimensional
 - ▶ the dimensionality of $e(w)$ is the size of the vocabulary
 - ▶ a typical vocabulary size is $\approx 100\,000$
 - ▶ a window of 10 words would correspond to at least $1\,000\,000$ units!

ONE-HOT ENCODING MODEL

- The major problem with the one-hot representation is that it is very high-dimensional
 - the dimensionality of $e(w)$ is the size of the vocabulary
 - a typical vocabulary size is $\approx 100\,000$
 - a window of 10 words would correspond to at least $1\,000\,000$ units!
- This has 2 consequences:
 - vulnerability to overfitting
 - computationally expensive

WORD REPRESENTATIONS

Topics: continuous word representation

- Idea: learn a continuous representation of words
- each word w is associated with a real-valued vector $C(w)$

Word	w	$C(w)$
" the "	1	[0.6762, -0.9607, 0.3626, -0.2410, 0.6636]
" a "	2	[0.6859, -0.9266, 0.3777, -0.2140, 0.6711]
" have "	3	[0.1656, -0.1530, 0.0310, -0.3321, -0.1342]
" be "	4	[0.1760, -0.1340, 0.0702, -0.2981, -0.1111]
" cat "	5	[0.5896, 0.9137, 0.0452, 0.7603, -0.6541]
" dog "	6	[0.5965, 0.9143, 0.0899, 0.7702, -0.6392]
" car "	7	[-0.0069, 0.7995, 0.6433, 0.2898, 0.6359]
...	...	...

1-hot vectors

How can we represent words as vectors?

Option 1: each element represents a different word.

Also known as “1-hot” or “1-of-V” representation.

	<i>bear</i>	<i>cat</i>	<i>frog</i>
bear	1	0	0
cat	0	1	0
frog	0	0	1

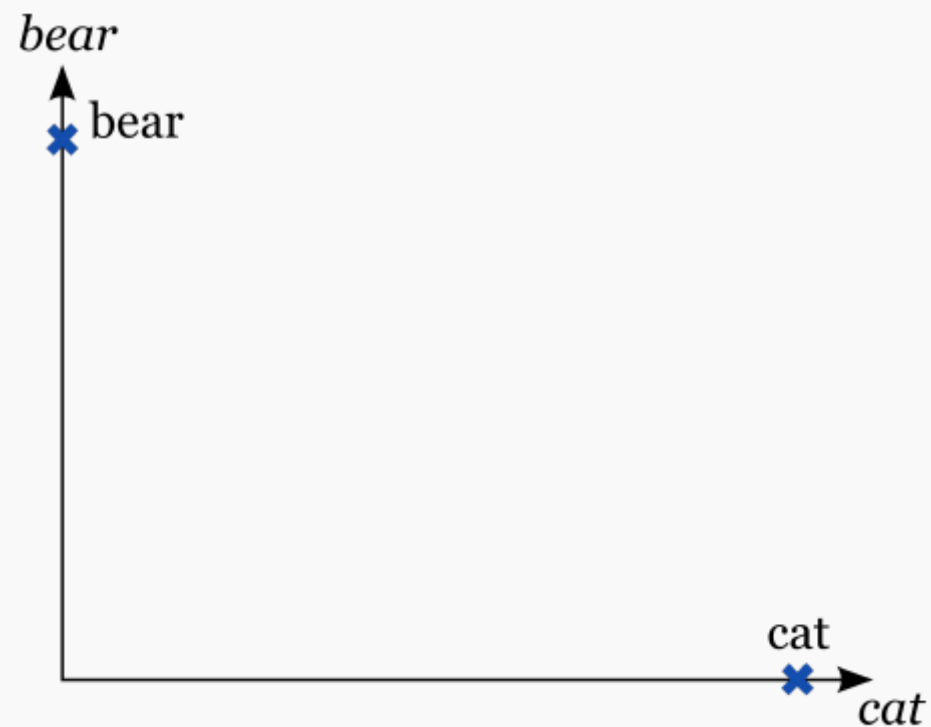
bear=[1.0, 0.0, 0.0]

cat=[0.0, 1.0, 0.0]

1-hot vectors

When using 1-hot vectors, we can't fit many and they tell us very little.

Need a separate dimension for every word we want to represent.



Distributed vectors

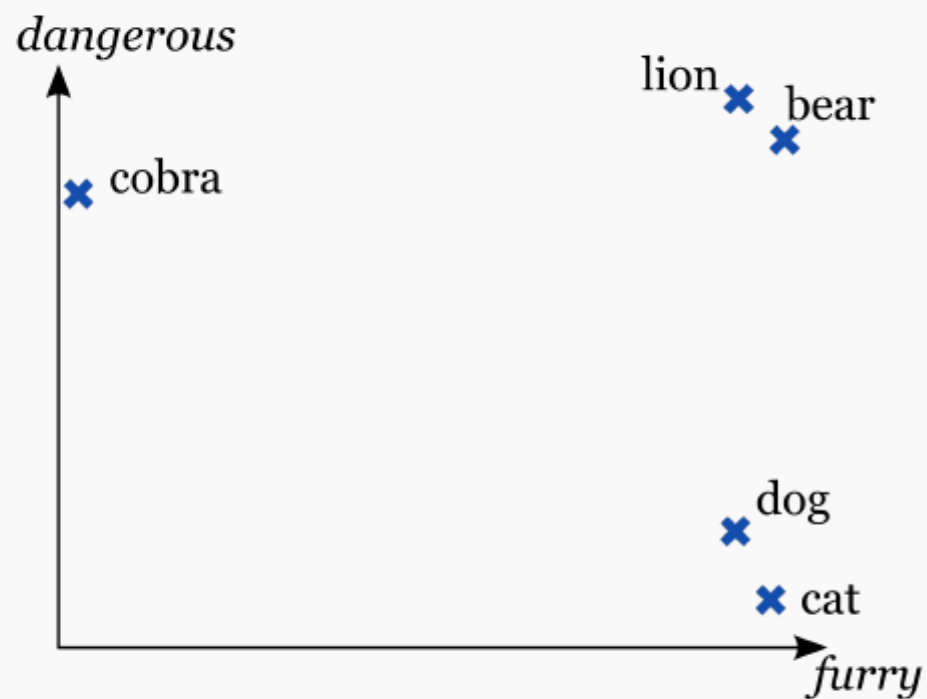
Option 2: each element represents a property, and they are shared between the words.

Also known as “distributed” representation.

	<i>furry</i>	<i>dangerous</i>	<i>mammal</i>
bear	0.9	0.85	1
cat	0.85	0.15	1
frog	0	0.05	0

bear = [0.9, 0.85, 1.0] cat = [0.85, 0.15, 1.0]

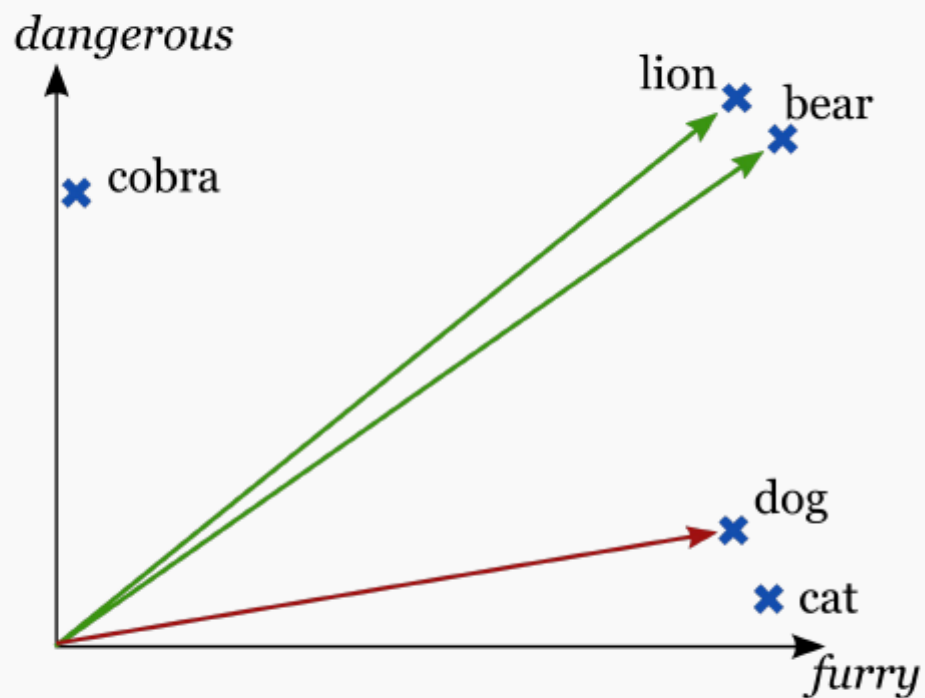
Distributed vectors



	<i>furry</i>	<i>dangerous</i>
bear	0.9	0.85
cat	0.85	0.15
cobra	0.0	0.8
lion	0.85	0.9
dog	0.8	0.15

Distributed vectors group similar words/objects together

Distributed vectors



$$\cos(a, b) = \frac{\sum_i a_i \cdot b_i}{\sqrt{\sum_i a_i^2} \cdot \sqrt{\sum_i b_i^2}}$$

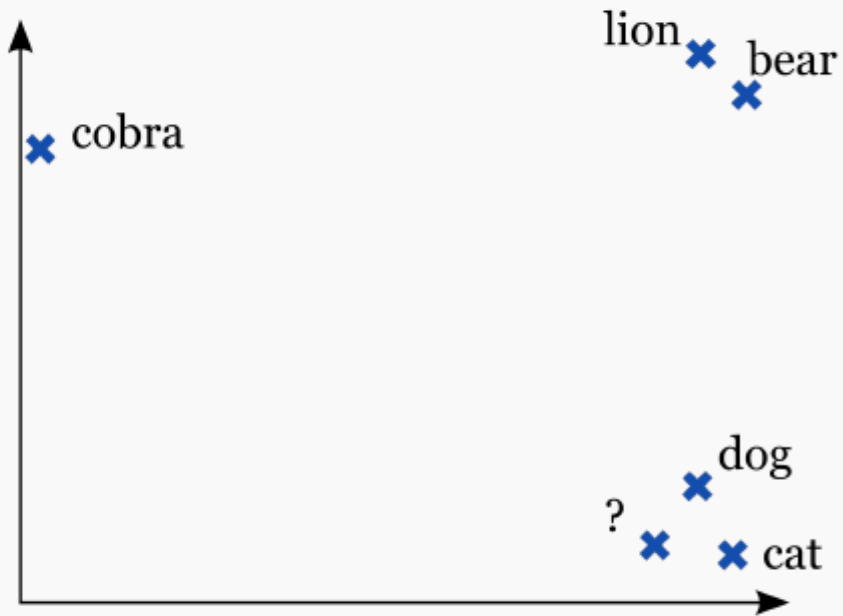
$$\cos(\text{lion}, \text{bear}) = 0.998$$

$$\cos(\text{lion}, \text{dog}) = 0.809$$

$$\cos(\text{cobra}, \text{dog}) = 0.727$$

Can use cosine to calculate similarity between two words

Distributed vectors



We can infer some information, based only on the vector of the word

We don't even need to know the labels on the vector elements

WORD REPRESENTATIONS

Topics: continuous word representation

- Idea: learn a continuous representation of words
 - we would like the distance $\|C(w) - C(w')\|$ to reflect meaningful similarities between words

Hierarchical Distributed Representations for Statistical Language Modeling

John Blitzer, Kilian Q. Weinberger, Lawrence K. Saul, and Fernando C. N. Pereira

Department of Computer and Information Science, University of Pennsylvania

Levine Hall, 3330 Walnut Street, Philadelphia, PA 19104

{blitzer,kilianw,lsaul,pereira}@cis.upenn.edu

Abstract

Statistical language models estimate the probability of a word occurring in a given context. The most common language models rely on a discrete enumeration of predictive contexts (e.g., n -grams) and consequently fail to capture and exploit statistical regularities across these contexts. In this paper, we show how to learn hierarchical, distributed representations of word contexts that maximize the predictive value of a statistical language model. The representations are initialized by unsupervised algorithms for

https://proceedings.neurips.cc/paper_files/paper/2004/file/110eec23201d80e40d0c4a48954e2ff5-Paper.pdf

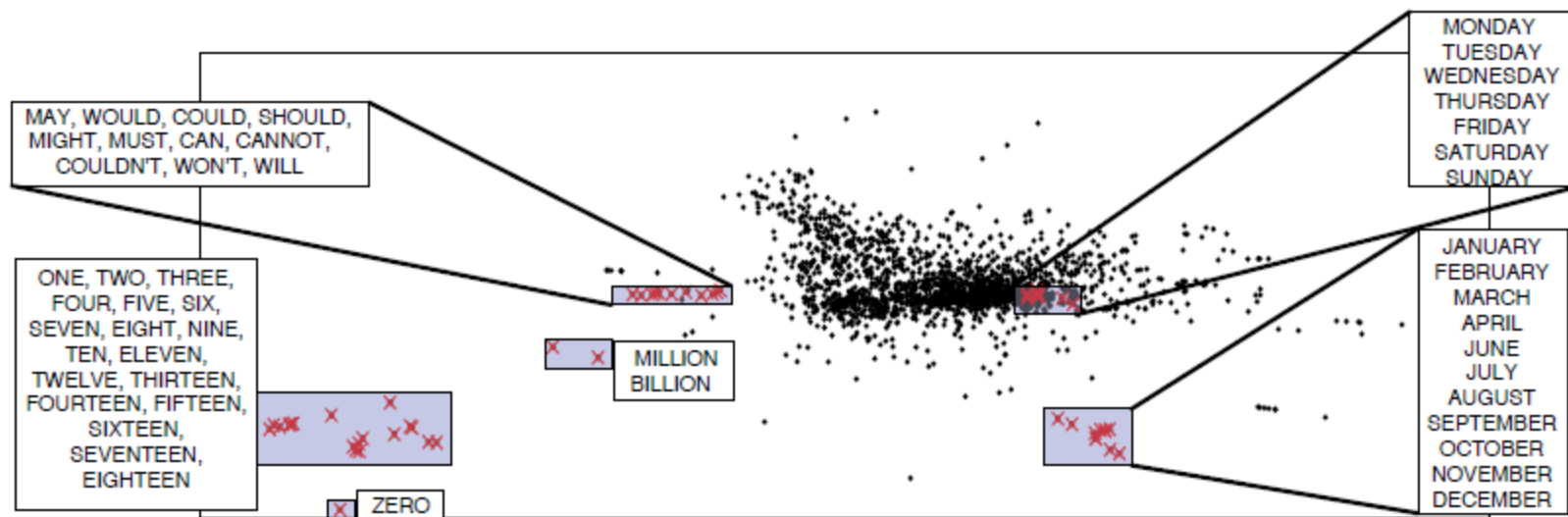


Figure 2: Projection of the normalized bigram counts of the 2000 most frequent words onto the first two dimensions of the nonlinear embedding obtained by semidefinite programming. Note that semantically meaningful neighborhoods are preserved, despite the massive dimensionality reduction from $V = 60000$ to $d = 2$.

Word representations: A simple and general method for semi-supervised learning

Joseph Turian

Département d'Informatique et
Recherche Opérationnelle (DIRO)
Université de Montréal

Montréal, Québec, Canada, H3T 1J4
lastname@iro.umontreal.ca

Lev Ratinov

Department of
Computer Science
University of Illinois at
Urbana-Champaign

Urbana, IL 61801
ratinov2@uiuc.edu

Yoshua Bengio

Département d'Informatique et
Recherche Opérationnelle (DIRO)
Université de Montréal

Montréal, Québec, Canada, H3T 1J4
bengioy@iro.umontreal.ca

Abstract

If we take an existing supervised NLP system, a simple and general way to improve accuracy is to use unsupervised word representations as extra word features. We evaluate Brown clusters, Collobert and Weston (2008) embeddings, and HLBL (Mnih & Hinton, 2009) embeddings of words on both NER and chunking. We use near state-of-the-art supervised baselines, and find that each of the three word representations improves the accuracy of these baselines. We find further improvements by combining different word representations. You can download our word features, for off-the-shelf use in existing NLP systems, as well as our code, here: <http://metaoptimize.com/projects/wordreprs/>

already been induced—plug these word features into an existing system, and observe a significant increase in accuracy. But which word features are good for what tasks? Should we prefer certain word features? Can we combine them?

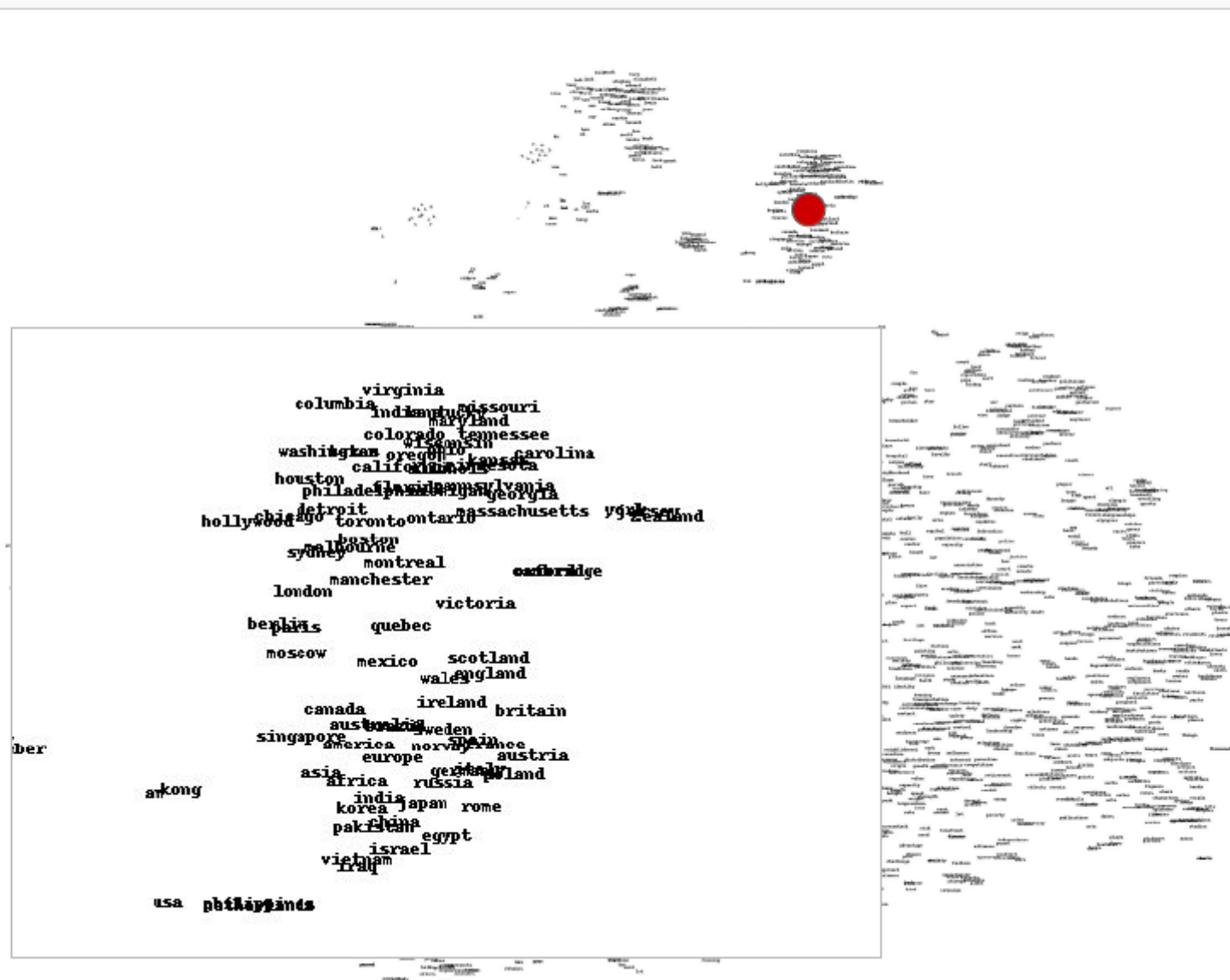
A *word representation* is a mathematical object associated with each word, often a vector. Each dimension's value corresponds to a feature and might even have a semantic or grammatical interpretation, so we call it a *word feature*. Conventionally, supervised lexicalized NLP approaches take a word and convert it to a symbolic ID, which is then transformed into a feature vector using a *one-hot* representation: The feature vector has the same length as the size of the vocabulary, and only one dimension is on. However, the one-hot representation of a word suffers from data sparsity: Namely, for words that are rare in the labeled training data, their corresponding model parameters will be poorly estimated. Moreover,

<https://aclanthology.org/P10-1040.pdf>

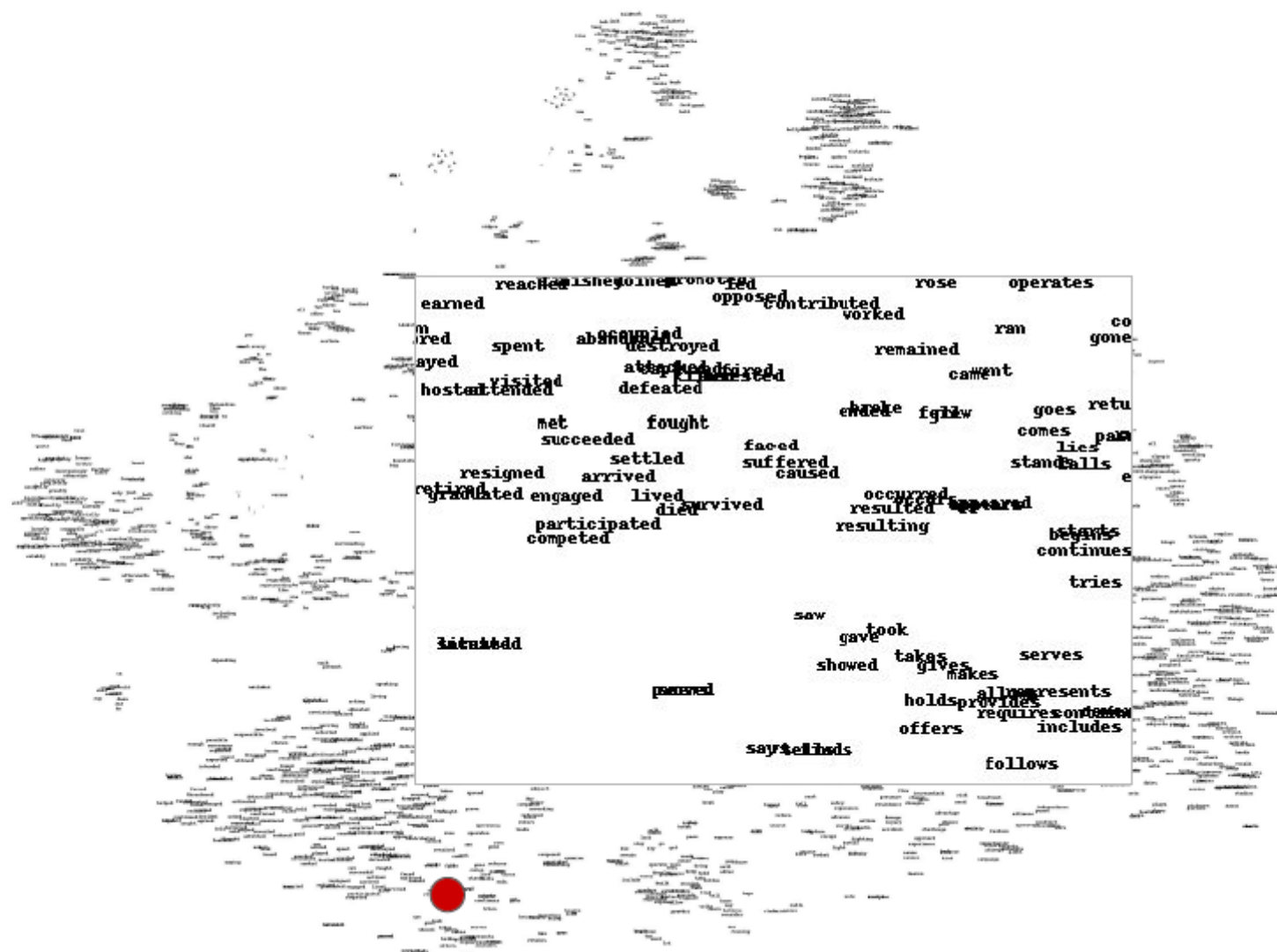
Word similarity



Word similarity



Word similarity



Joseph Turian

Language Modeling

Language Modeling

Task: given a sequence of words, predict the next word.

the cat sat on the ??

Language Modeling

Task: given a sequence of words, predict the next word.

the cat sat on the ??



Showing results for **the cat sat on the mat**

Search instead for "the cat sat on"

LANGUAGE MODELING

Topics: language modeling

- A language model is a probabilistic model that assigns probabilities to any sequence of words

$$p(w_1, \dots, w_T)$$

- Goal: compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$$

- Goal: compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$$

- Related task: probability of an upcoming word:

$$P(w_5 | w_1, w_2, w_3, w_4)$$

- Goal: compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$$

- Related task: probability of an upcoming word:

$$P(w_5 | w_1, w_2, w_3, w_4)$$

- A model that computes either of these:

$P(W)$ or $P(w_n | w_1, w_2 \dots w_{n-1})$ is called a **language model**.

$$P(B|A) = \frac{P(A,B)}{P(A)}$$



$$P(A, B) = P(A)P(B|A)$$

$$P(B|A) = \frac{P(A,B)}{P(A)}$$



$$P(A, B) = P(A)P(B|A)$$

$$P(C|A, B) = \frac{P(A, B, C)}{P(A, B)}$$



$$\begin{aligned} P(A, B, C) &= P(A, B)P(C|A, B) \\ &= P(A)P(B|A)P(C|A, B) \end{aligned}$$

$$P(B|A) = \frac{P(A,B)}{P(A)}$$



$$P(A, B) = P(A)P(B|A)$$

$$P(C|A, B) = \frac{P(A, B, C)}{P(A, B)}$$



$$\begin{aligned} P(A, B, C) &= P(A, B)P(C|A, B) \\ &= P(A)P(B|A)P(C|A, B) \end{aligned}$$

- The Chain Rule in General

$$P(x_1, x_2, x_3, \dots, x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2) \dots P(x_n|x_1, \dots, x_{n-1})$$

$$P(B|A) = \frac{P(A,B)}{P(A)}$$



$$P(A, B) = P(A)P(B|A)$$

$$P(C|A, B) = \frac{P(A, B, C)}{P(A, B)}$$



$$\begin{aligned} P(A, B, C) &= P(A, B)P(C|A, B) \\ &= P(A)P(B|A)P(C|A, B) \end{aligned}$$

- The Chain Rule in General

$$P(x_1, x_2, x_3, \dots, x_n) = P(x_1)P(x_2 | x_1)P(x_3 | x_1, x_2) \dots P(x_n | x_1, \dots, x_{n-1})$$

Activate
Go to Setti

$$P(w_1 w_2 \dots w_n) = \prod_i P(w_i | w_1 w_2 \dots w_{i-1})$$

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

sentence beginning

sentence ending

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

sentence beginning
sentence ending

<s> I am Sam </s>
 <s> Sam I am </s>
 <s> I do not like green eggs and ham </s>

$P(\text{I} \text{<s>}) = \frac{2}{3} = .67$	$P(\text{Sam} \text{<s>}) = \frac{1}{3} = .33$	$P(\text{am} \text{I}) = \frac{2}{3} = .67$
$P(\text{</s>} \text{Sam}) = \frac{1}{2} = 0.5$	$P(\text{Sam} \text{am}) = \frac{1}{2} = .5$	$P(\text{do} \text{I}) = \frac{1}{3} = .33$

$$P(w_1w_2\dots w_n) = \prod_i P(w_i \mid w_1w_2\dots w_{i-1})$$

$$P(w_1 w_2 \dots w_n) = \prod_i P(w_i \mid w_1 w_2 \dots w_{i-1})$$

$$P(w_i \mid w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_{i-1}, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$$

- Could we just count and divide?

$$P(w_1 w_2 \dots w_n) = \prod_i P(w_i \mid w_1 w_2 \dots w_{i-1})$$

$$P(w_i \mid w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_{i-1}, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$$

- Could we just count and divide?
- No! Too many possible sentences!

$$P(w_1 w_2 \dots w_n) = \prod_i P(w_i \mid w_1 w_2 \dots w_{i-1})$$

$$P(w_i \mid w_1, w_2, \dots, w_{i-1}) = \frac{P(w_1, w_2, \dots, w_{i-1}, w_i)}{P(w_1, w_2, \dots, w_{i-1})}$$

- Could we just count and divide?
- No! Too many possible sentences!
- We'll never see enough data for estimating these

Markov Assumption

$$P(w_1 w_2 \dots w_n) = \prod_i P(w_i \mid w_1 w_2 \dots w_{i-1})$$

Andrey Andreyevich Markov



A. A. Markov (1886).

Markov in 1886

Born	14 June 1856 N.S. Ryazan, Russian Empire
Died	20 July 1922 (aged 66) Petrograd, Russian SFSR

https://en.wikipedia.org/wiki/Andrey_Markov

Markov Assumption

$$P(w_1 w_2 \dots w_n) = \prod_i P(w_i \mid w_1 w_2 \dots w_{i-1})$$
$$\approx \prod_i P(w_i \mid w_{i-k} \dots w_{i-1})$$

Andrey Andreyevich Markov



A. A. Markov (1886).

Markov in 1886

Born	14 June 1856 N.S. Ryazan, Russian Empire
Died	20 July 1922 (aged 66) Petrograd, Russian SFSR

https://en.wikipedia.org/wiki/Andrey_Markov

Markov Assumption

$$P(w_1 w_2 \dots w_n) = \prod_i P(w_i \mid w_1 w_2 \dots w_{i-1})$$
$$\approx \prod_i P(w_i \mid w_{i-k} \dots w_{i-1})$$

$$P(w_i \mid w_1 w_2 \dots w_{i-1}) \approx P(w_i \mid w_{i-k} \dots w_{i-1})$$

Andrey Andreyevich Markov



A. A. Markov (1886).

Markov in 1886

Born	14 June 1856 N.S. Ryazan, Russian Empire
Died	20 July 1922 (aged 66) Petrograd, Russian SFSR

https://en.wikipedia.org/wiki/Andrey_Markov

Language Modeling

- You can also think of a Language Model as a system that assigns a probability to a piece of text

Language Modeling

- You can also think of a Language Model as a system that assigns a probability to a piece of text
- For example, if we have some text $x^{(1)}, \dots, x^{(T)}$, then the probability of this text (according to the Language Model) is:

Language Modeling

- You can also think of a Language Model as a system that assigns a probability to a piece of text
- For example, if we have some text $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}$, then the probability of this text (according to the Language Model) is:

$$\begin{aligned} P(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}) &= P(\mathbf{x}^{(1)}) \times P(\mathbf{x}^{(2)} | \mathbf{x}^{(1)}) \times \dots \times P(\mathbf{x}^{(T)} | \mathbf{x}^{(T-1)}, \dots, \mathbf{x}^{(1)}) \\ &= \prod_{t=1}^T P(\mathbf{x}^{(t)} | \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)}) \end{aligned}$$

Language Modeling

- You can also think of a Language Model as a system that assigns a probability to a piece of text
- For example, if we have some text $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}$, then the probability of this text (according to the Language Model) is:

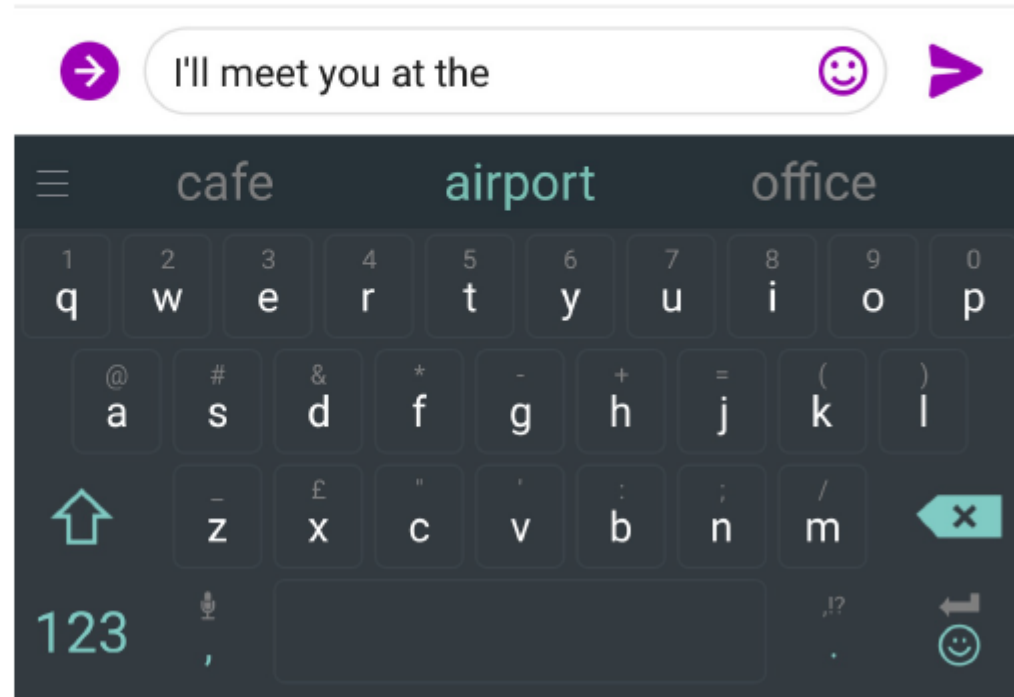
$$\begin{aligned} P(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}) &= P(\mathbf{x}^{(1)}) \times P(\mathbf{x}^{(2)} | \mathbf{x}^{(1)}) \times \dots \times P(\mathbf{x}^{(T)} | \mathbf{x}^{(T-1)}, \dots, \mathbf{x}^{(1)}) \\ &= \prod_{t=1}^T P(\mathbf{x}^{(t)} | \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)}) \end{aligned}$$


This is what our LM provides

<https://www.youtube.com/watch?v=iWea12EAu6U&list=PLoROMvodv4rOhcuXMZkNm7j3fVwBBY42z&index=6>

<https://web.stanford.edu/class/cs224n/>

You use Language Models every day!



You use Language Models every day!



what is the | 

what is the **weather**
what is the **meaning of life**
what is the **dark web**
what is the **xfi**
what is the **doomsday clock**
what is the **weather today**
what is the **keto diet**
what is the **american dream**
what is the **speed of light**
what is the **bill of rights**

Google Search I'm Feeling Lucky

n-gram Language Models

?

n-gram Language Models

the students opened their _____?

n-gram Language Models

the students opened their _____?

- **Question:** How to learn a Language Model?
- **Answer** (pre- Deep Learning): learn an *n*-gram Language Model!

n-gram Language Models

the students opened their _____?

- **Question:** How to learn a Language Model?
- **Answer** (pre- Deep Learning): learn an *n*-gram Language Model!
- **Definition:** An *n*-gram is a chunk of *n* consecutive words.

n-gram Language Models

the students opened their _____?

- **Question:** How to learn a Language Model?
- **Answer** (pre- Deep Learning): learn an *n*-gram Language Model!
- **Definition:** An *n*-gram is a chunk of *n* consecutive words.
 - **unigrams:** “the”, “students”, “opened”, “their”
 - **bigrams:** “the students”, “students opened”, “opened their”
 - **trigrams:** “the students opened”, “students opened their”
 - **four-grams:** “the students opened their”

n-gram Language Models

the students opened their _____?

- **Question:** How to learn a Language Model?
- **Answer** (pre- Deep Learning): learn an *n*-gram Language Model!
- **Definition:** An *n*-gram is a chunk of *n* consecutive words.
 - **unigrams:** "the", "students", "opened", "their"
 - **bigrams:** "the students", "students opened", "opened their"
 - **trigrams:** "the students opened", "students opened their"
 - **four-grams:** "the students opened their"
- **Idea:** Collect statistics about how frequent different n-grams are and use these to predict next word.

Simplest case: Unigram model

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i)$$

Bigram model

- Condition on the previous word:

$$P(w_i \mid w_1 w_2 \dots w_{i-1}) \approx P(w_i \mid w_{i-1})$$

•
•
•

Estimating bigram probabilities

- The Maximum Likelihood Estimate

$$P(w_i \mid w_{i-1}) = \frac{\textit{count}(w_{i-1}, w_i)}{\textit{count}(w_{i-1})}$$

Estimating bigram probabilities

- The Maximum Likelihood Estimate

$$P(w_i | w_{i-1}) = \frac{\textit{count}(w_{i-1}, w_i)}{\textit{count}(w_{i-1})}$$

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green eggs and ham </s>

$$P(\text{I} | \text{<s>}) = \frac{2}{3} = .67$$

$$P(\text{Sam} | \text{<s>}) = \frac{1}{3} = .33$$

$$P(\text{am} | \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} | \text{Sam}) = \frac{1}{2} = 0.5$$

$$P(\text{Sam} | \text{am}) = \frac{1}{2} = .5$$

$$P(\text{do} | \text{I}) = \frac{1}{3} = .33$$

n-gram Language Models

- First we make a **Markov assumption**: $x^{(t+1)}$ depends only on the preceding $n-1$ words

$$P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)}) = P(\mathbf{x}^{(t+1)} | \overbrace{\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}}^{n-1 \text{ words}}) \quad (\text{assumption})$$

n-gram Language Models

- First we make a **Markov assumption**: $x^{(t+1)}$ depends only on the preceding $n-1$ words

$$P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)}) = P(x^{(t+1)} | \overbrace{x^{(t)}, \dots, x^{(t-n+2)}}^{n-1 \text{ words}}) \quad (\text{assumption})$$

prob of a n-gram

$$= P(x^{(t+1)}, x^{(t)}, \dots, x^{(t-n+2)})$$

prob of a (n-1)-gram

$$P(x^{(t)}, \dots, x^{(t-n+2)})$$

(definition of
conditional prob)

n-gram Language Models

- First we make a **Markov assumption**: $x^{(t+1)}$ depends only on the preceding $n-1$ words

$$P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)}) = P(x^{(t+1)} | \overbrace{x^{(t)}, \dots, x^{(t-n+2)}}^{n-1 \text{ words}}) \quad (\text{assumption})$$

prob of a n-gram

$$= P(x^{(t+1)}, x^{(t)}, \dots, x^{(t-n+2)})$$

prob of a (n-1)-gram

$$P(x^{(t)}, \dots, x^{(t-n+2)})$$

(definition of
conditional prob)

- Question:** How do we get these n -gram and $(n-1)$ -gram probabilities?

n-gram Language Models

- First we make a **Markov assumption**: $x^{(t+1)}$ depends only on the preceding $n-1$ words

$$P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)}) = P(x^{(t+1)} | \overbrace{x^{(t)}, \dots, x^{(t-n+2)}}^{n-1 \text{ words}}) \quad (\text{assumption})$$

prob of a n-gram $\rightarrow$

prob of a (n-1)-gram $\rightarrow$

$$= \frac{P(x^{(t+1)}, x^{(t)}, \dots, x^{(t-n+2)})}{P(x^{(t)}, \dots, x^{(t-n+2)})} \quad (\text{definition of conditional prob})$$

- Question:** How do we get these n -gram and $(n-1)$ -gram probabilities?
- Answer:** By **counting** them in some large corpus of text!

$$\approx \frac{\text{count}(x^{(t+1)}, x^{(t)}, \dots, x^{(t-n+2)})}{\text{count}(x^{(t)}, \dots, x^{(t-n+2)})} \quad (\text{statistical approximation})$$

n-gram Language Models: Example

the students opened their [?]_____

n-gram Language Models: Example

the students opened their _____[?]

Suppose we are learning a 4-gram Language Model.

~~as the proctor started the clock, the~~ *students opened their _____*
discard { condition on this

n-gram Language Models: Example

the students opened their _____ ?

Suppose we are learning a 4-gram Language Model.

~~as the proctor started the clock, the~~ *students opened their _____*
discard { condition on this

$$P(\boldsymbol{w} | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

n-gram Language Models: Example

the students opened their _____[?]

Suppose we are learning a 4-gram Language Model.

~~as the proctor started the clock, the~~ students opened their _____
discard condition on this

$$P(\mathbf{w} | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \mathbf{w})}{\text{count}(\text{students opened their})}$$

For example, suppose that in the corpus:

- “students opened their” occurred 1000 times
- “students opened their books” occurred 400 times
 - $\rightarrow P(\text{books} | \text{students opened their}) = 0.4$
- “students opened their exams” occurred 100 times
 - $\rightarrow P(\text{exams} | \text{students opened their}) = 0.1$

n-gram Language Models: Example

the students opened their _____[?]

Suppose we are learning a 4-gram Language Model.

~~as the proctor started the clock, the~~ students opened their _____
discard condition on this

$$P(\mathbf{w} | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \mathbf{w})}{\text{count}(\text{students opened their})}$$

For example, suppose that in the corpus:

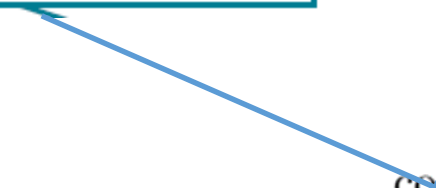
- “students opened their” occurred 1000 times
- “students opened their books” occurred 400 times
 - $\rightarrow P(\text{books} | \text{students opened their}) = 0.4$
- “students opened their exams” occurred 100 times
 - $\rightarrow P(\text{exams} | \text{students opened their}) = 0.1$

Should we have discarded the “proctor” context?

Sparsity Problems with n-gram Language Models

Sparsity Problem 1

Problem: What if “*students opened their w* ” never occurred in data? Then w has probability 0!

$$P(\boldsymbol{w} | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$


Sparsity Problems with n-gram Language Models

Sparsity Problem 1

Problem: What if “students opened their w ” never occurred in data? Then w has probability 0!

(Partial) Solution: Add small δ to the count for every $w \in V$. This is called *smoothing*.

$$P(w|\text{students opened their}) = \frac{\text{count}(\text{students opened their } w)}{\text{count}(\text{students opened their})}$$

Sparsity Problems with n-gram Language Models

Sparsity Problem 1

Problem: What if “students opened their w ” never occurred in data? Then w has probability 0!

(Partial) Solution: Add small δ to the count for every $w \in V$. This is called *smoothing*.

$$P(w|\text{students opened their}) = \frac{\text{count}(\text{students opened their } w)}{\text{count}(\text{students opened their})}$$

Sparsity Problem 2

Problem: What if “students opened their” never occurred in data? Then we can’t calculate probability for *any* w !

Sparsity Problems with n-gram Language Models

Sparsity Problem 1

Problem: What if “students opened their w ” never occurred in data? Then w has probability 0!

(Partial) Solution: Add small δ to the count for every $w \in V$. This is called *smoothing*.

$$P(w|\text{students opened their}) = \frac{\text{count}(\text{students opened their } w)}{\text{count}(\text{students opened their})}$$

Sparsity Problem 2

Problem: What if “students opened their” never occurred in data? Then we can’t calculate probability for *any* w !

(Partial) Solution: Just condition on “opened their” instead. This is called *backoff*.

Note: Increasing n makes sparsity problems *worse*. Typically, we can’t have n bigger than 5.

n-gram Language Models in practice

- You can build a simple trigram Language Model over a 1.7 million word corpus (Reuters) in a few seconds on your laptop

n-gram Language Models in practice

- You can build a simple trigram Language Model over a 1.7 million word corpus (Reuters) in a few seconds on your laptop

today the _____

n-gram Language Models in practice

- You can build a simple trigram Language Model over a 1.7 million word corpus (Reuters) in a few seconds on your laptop

today the _____

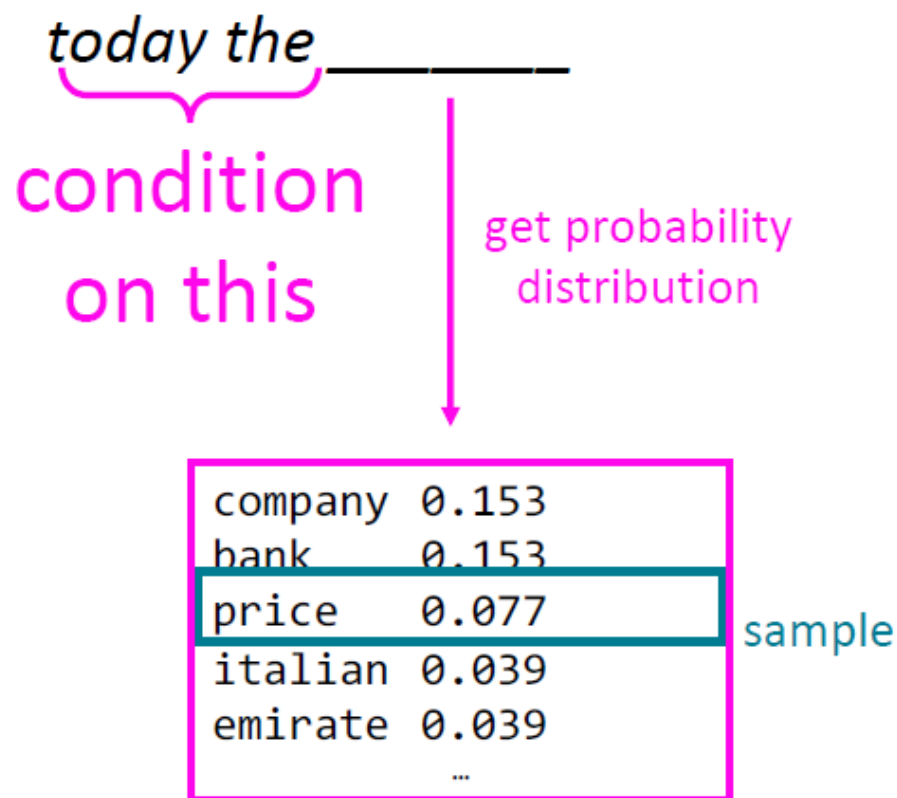
get probability
distribution

company	0.153
bank	0.153
price	0.077
italian	0.039
emirate	0.039
...	

Sparsity problem:
not much granularity
in the probability
distribution

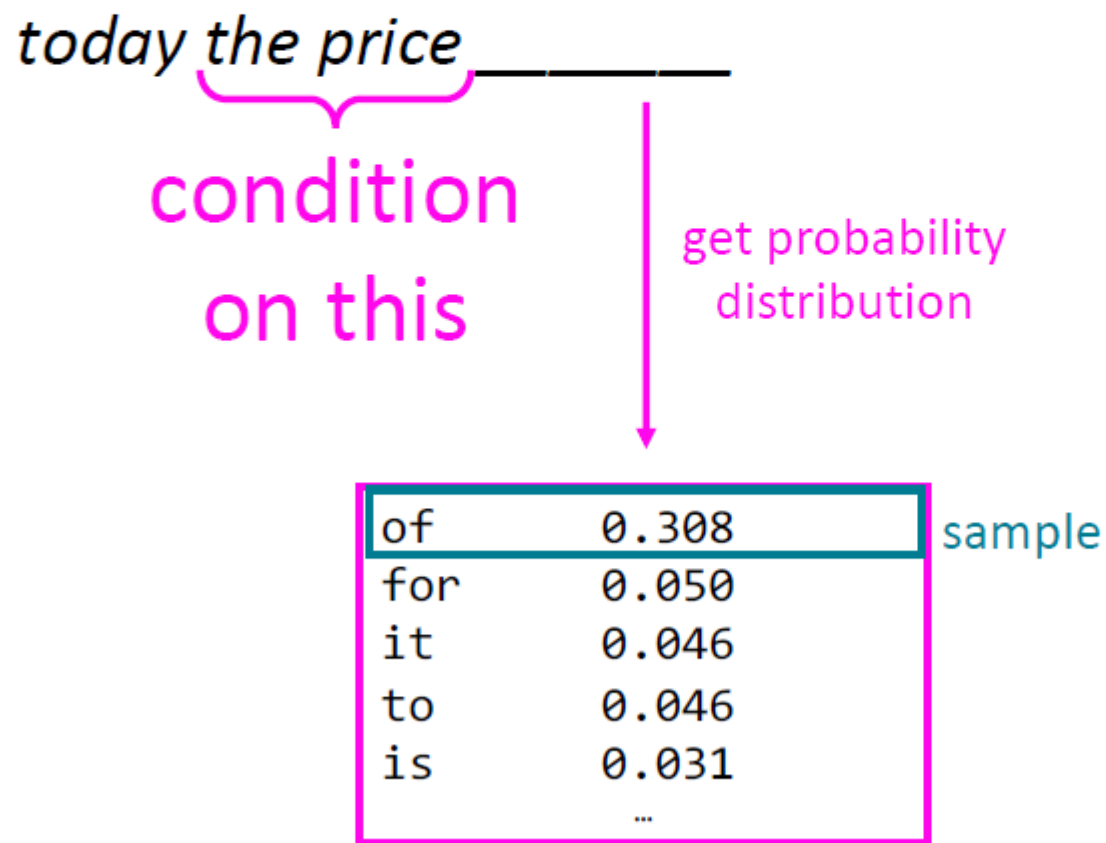
Generating text with a n-gram Language Model

You can also use a Language Model to generate text



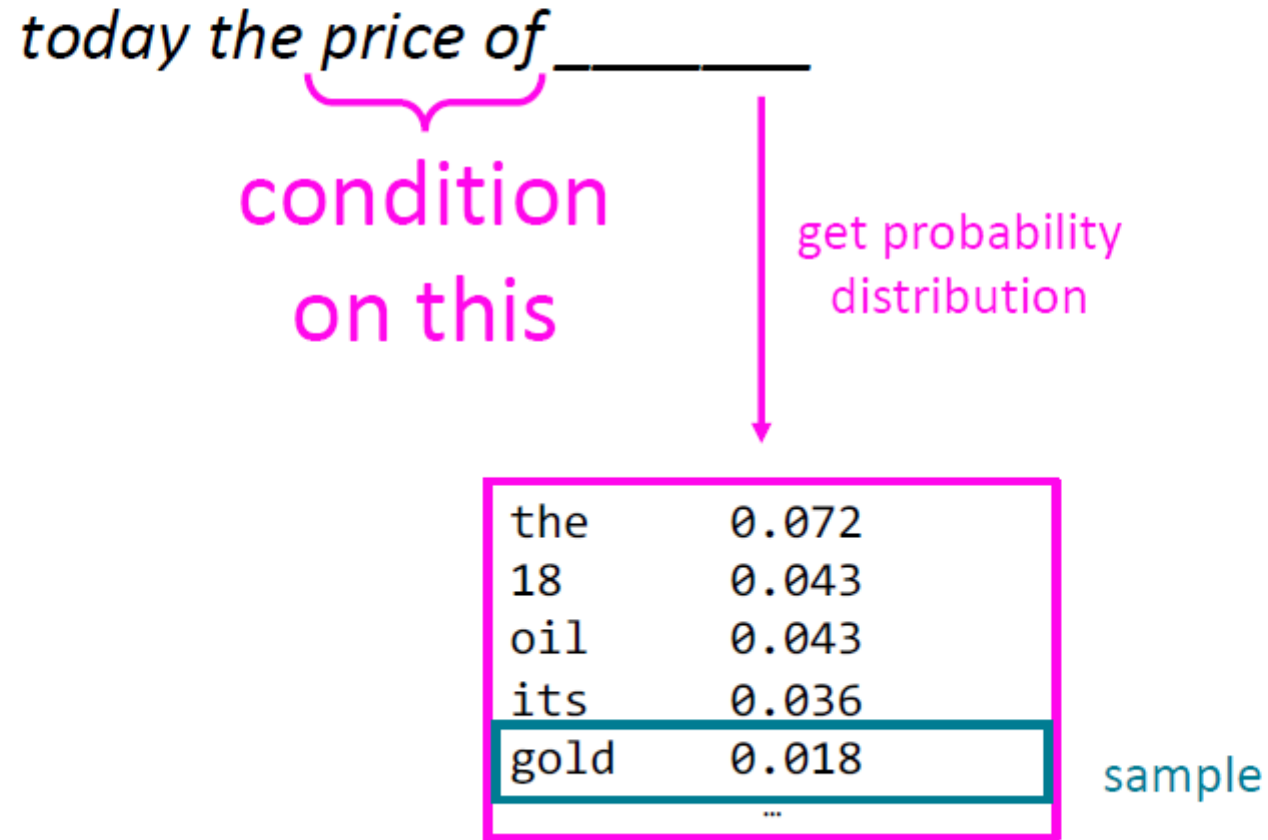
Generating text with a n-gram Language Model

You can also use a Language Model to generate text



Generating text with a n-gram Language Model

You can also use a Language Model to generate text



Generating text with a n-gram Language Model

You can also use a Language Model to generate text

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

Generating text with a n-gram Language Model

You can also use a Language Model to generate text

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

Surprisingly grammatical!

Generating text with a n-gram Language Model

You can also use a Language Model to generate text

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

Surprisingly grammatical!

...but **incoherent**. We need to consider more than three words at a time if we want to model language well.

Generating text with a n-gram Language Model

You can also use a Language Model to generate text

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

Surprisingly grammatical!

...but **incoherent**. We need to consider more than three words at a time if we want to model language well.

But increasing n worsens sparsity problem,
and increases model size...



The Unreasonable Effectiveness of Recurrent Neural Networks

May 21, 2015

There's something magical about Recurrent Neural Networks (RNNs). I still remember when I trained my first

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

N-Gram Language Modelling with NLTK

[Read](#)[Discuss](#)[Courses](#)[Video](#)[⋮](#)

Language modeling is the way of determining the probability of any sequence of words. Language modeling is used in a wide variety of applications such as Speech Recognition, Spam filtering, etc. In fact, language modeling is the key aim behind the implementation of many state-of-the-art Natural Language Processing models.

Methods of Language Modelings:

Two types of Language Modelings:

- **Statistical Language Modelings:** Statistical Language Modeling, or Language Modeling, is the development of probabilistic models that are able to predict the next word in the sequence given the words that precede. Examples such as N-gram language modeling.

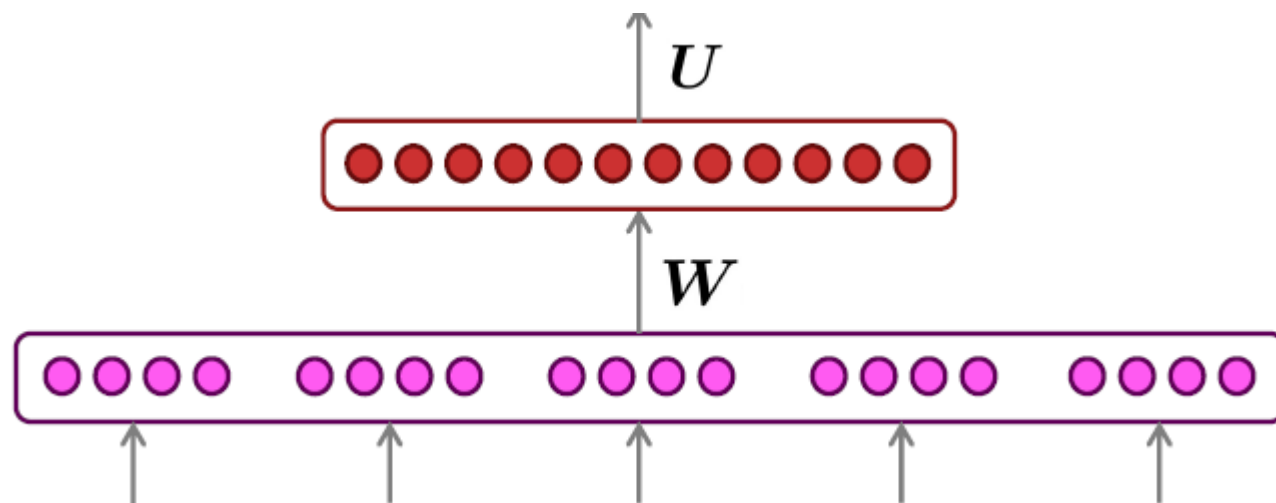
<https://www.geeksforgeeks.org/n-gram-language-modelling-with-nltk/>

Why the ngram model is silly...

- Suppose we have seen the sentence
“the cat walked in the garden on friday”
- This should help us predict words in the sentence
“the dog ran in the yard on monday”
- The model does not capture similarities
 - cat/dog squashed/flattened garden/yard friday/monday

How to build a *neural* language model?

- Recall the Language Modeling task:
 - Input: sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$
 - Output: prob. dist. of the next word $P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)})$



A fixed-window neural Language Model

output distribution

$$\hat{y} = \text{softmax}(U\mathbf{h} + \mathbf{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

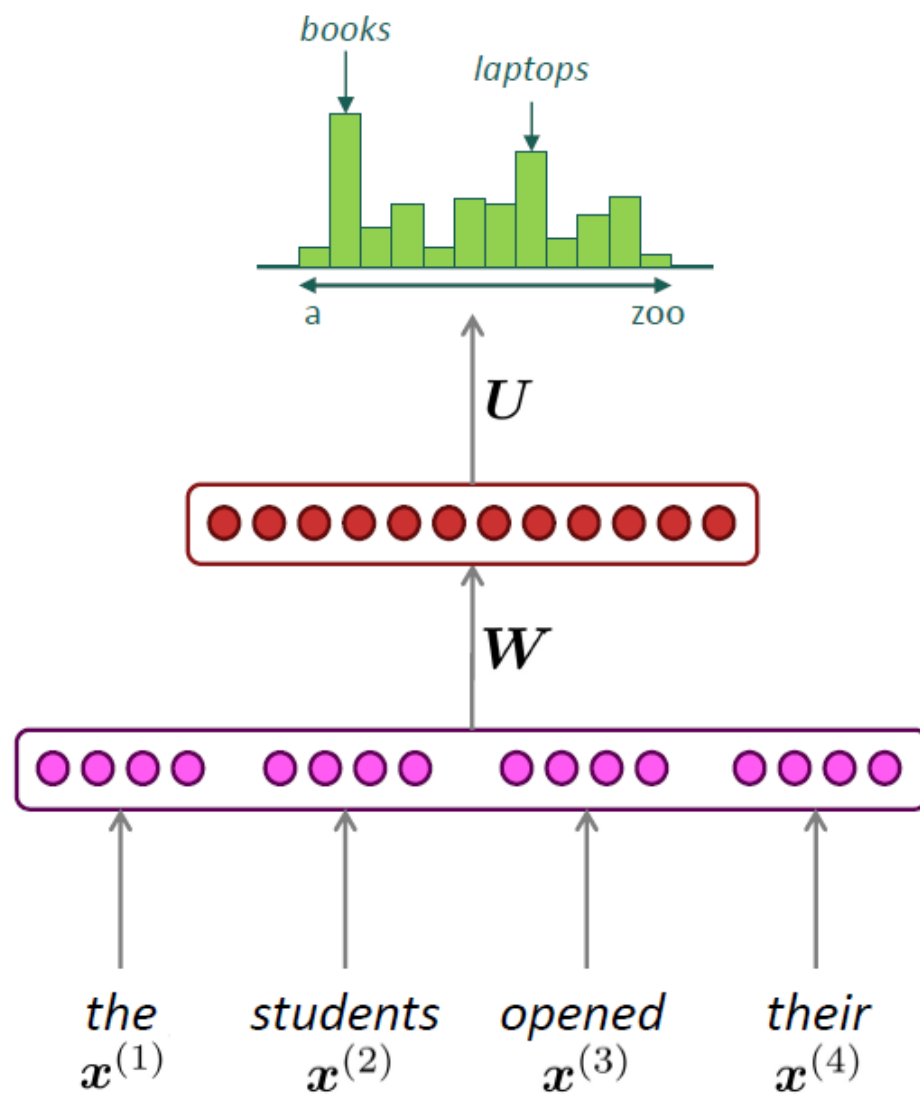
$$\mathbf{h} = f(\mathbf{W}\mathbf{e} + \mathbf{b}_1)$$

concatenated word embeddings

$$\mathbf{e} = [\mathbf{e}^{(1)}; \mathbf{e}^{(2)}; \mathbf{e}^{(3)}; \mathbf{e}^{(4)}]$$

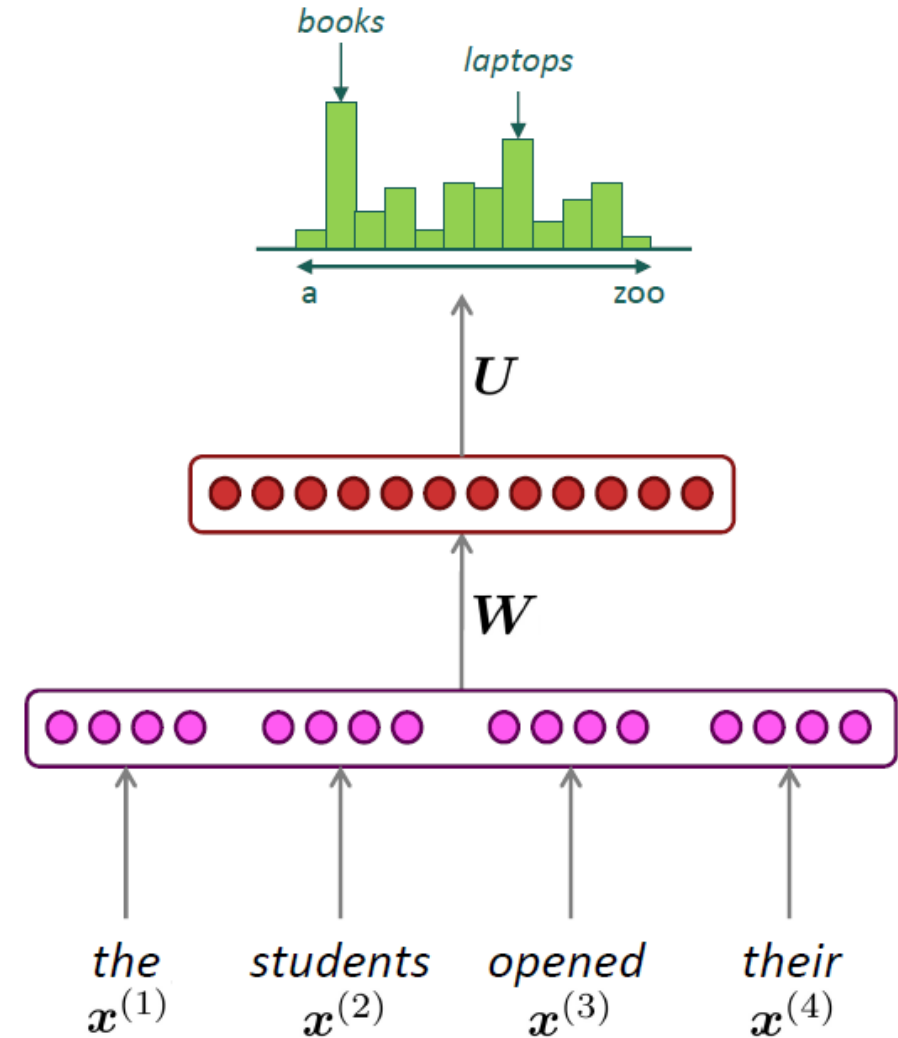
words / one-hot vectors

$$\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}, \mathbf{x}^{(4)}$$



A fixed-window neural Language Model

Approximately: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

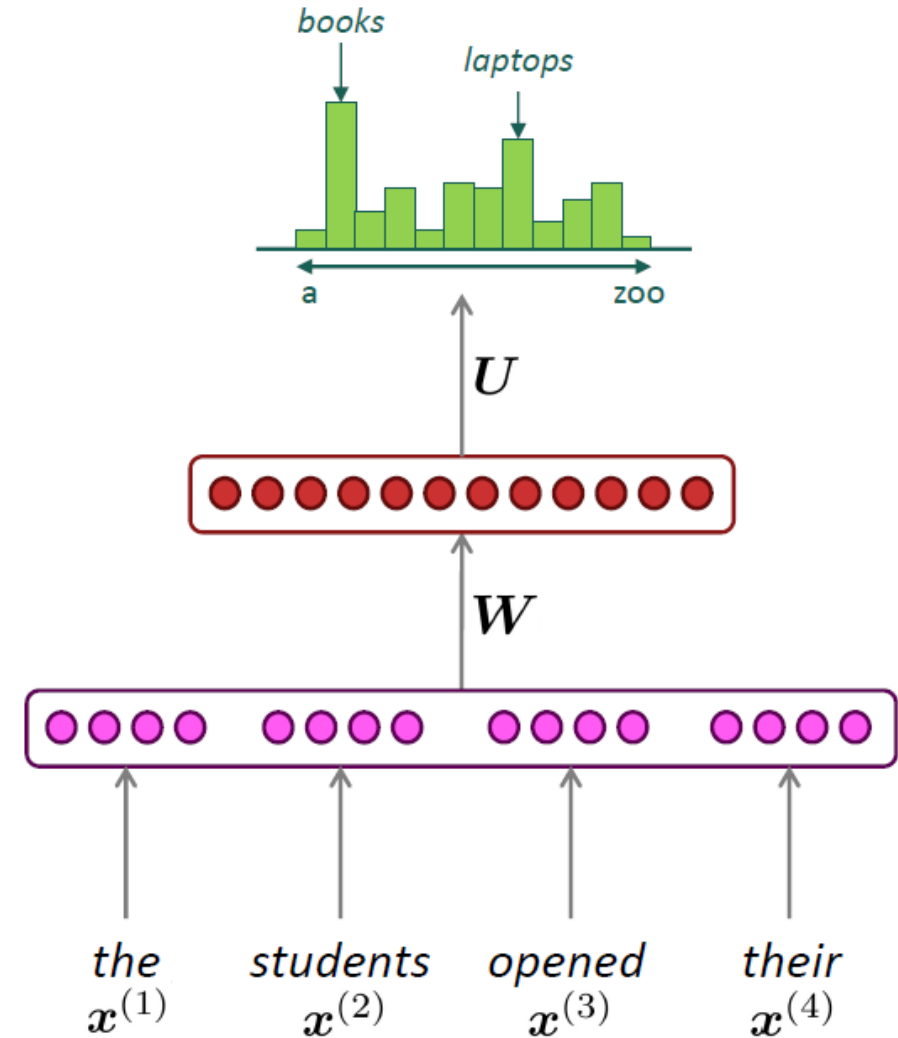


A fixed-window neural Language Model

Approximately: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

Improvements over n -gram LM:

- No sparsity problem
- Don't need to store all observed n -grams



A fixed-window neural Language Model

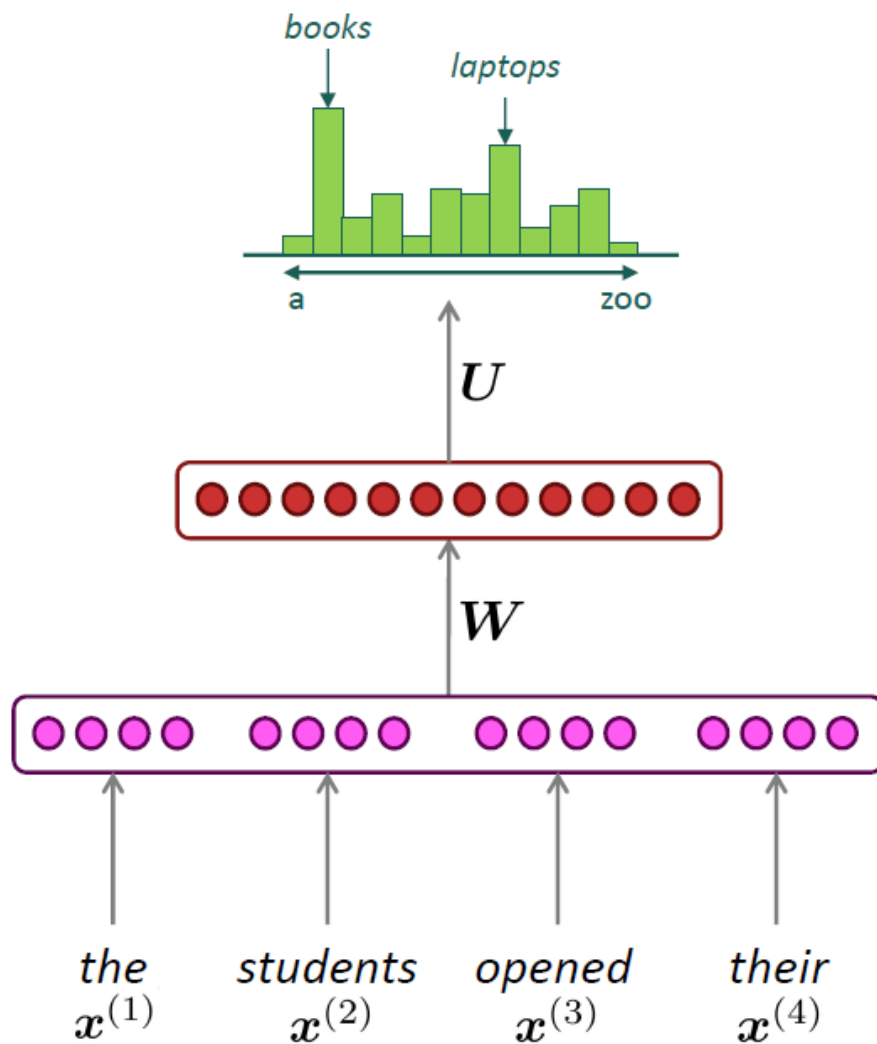
Approximately: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

Improvements over n -gram LM:

- No sparsity problem
- Don't need to store all observed n -grams

Remaining **problems**:

- Fixed window is **too small**
- Enlarging window enlarges W
- Window can never be large enough!
- $x^{(1)}$ and $x^{(2)}$ are multiplied by completely different weights in W .
No symmetry in how the inputs are processed.



A fixed-window neural Language Model

Approximately: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

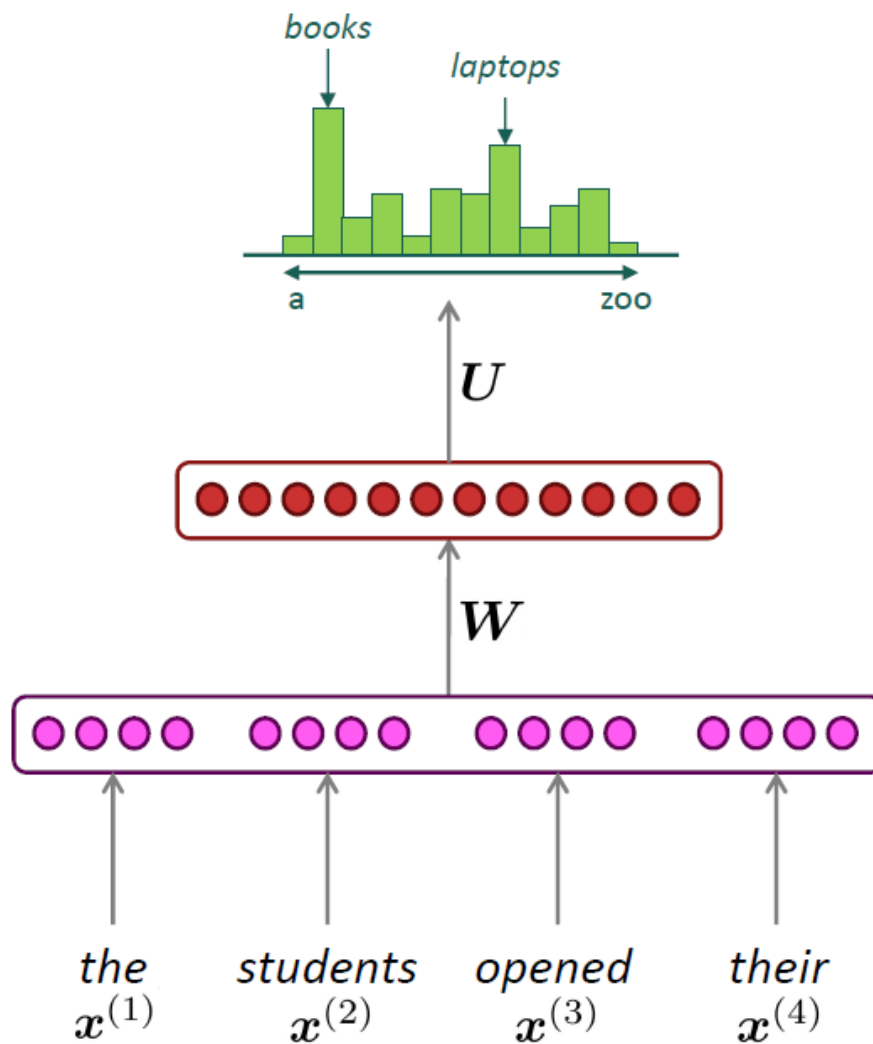
Improvements over n -gram LM:

- No sparsity problem
- Don't need to store all observed n -grams

Remaining **problems**:

- Fixed window is **too small**
- Enlarging window enlarges W
- Window can never be large enough!
- $x^{(1)}$ and $x^{(2)}$ are multiplied by completely different weights in W .
No symmetry in how the inputs are processed.

We need a neural architecture
that can process *any length input*



A Neural Probabilistic Language Model

Yoshua Bengio

Réjean Ducharme

Pascal Vincent

Christian Jauvin

Département d'Informatique et Recherche Opérationnelle

Centre de Recherche Mathématiques

Université de Montréal, Montréal, Québec, Canada

BENGIOY@IRO.UMONTREAL.CA

DUCHARME@IRO.UMONTREAL.CA

VINCENTP@IRO.UMONTREAL.CA

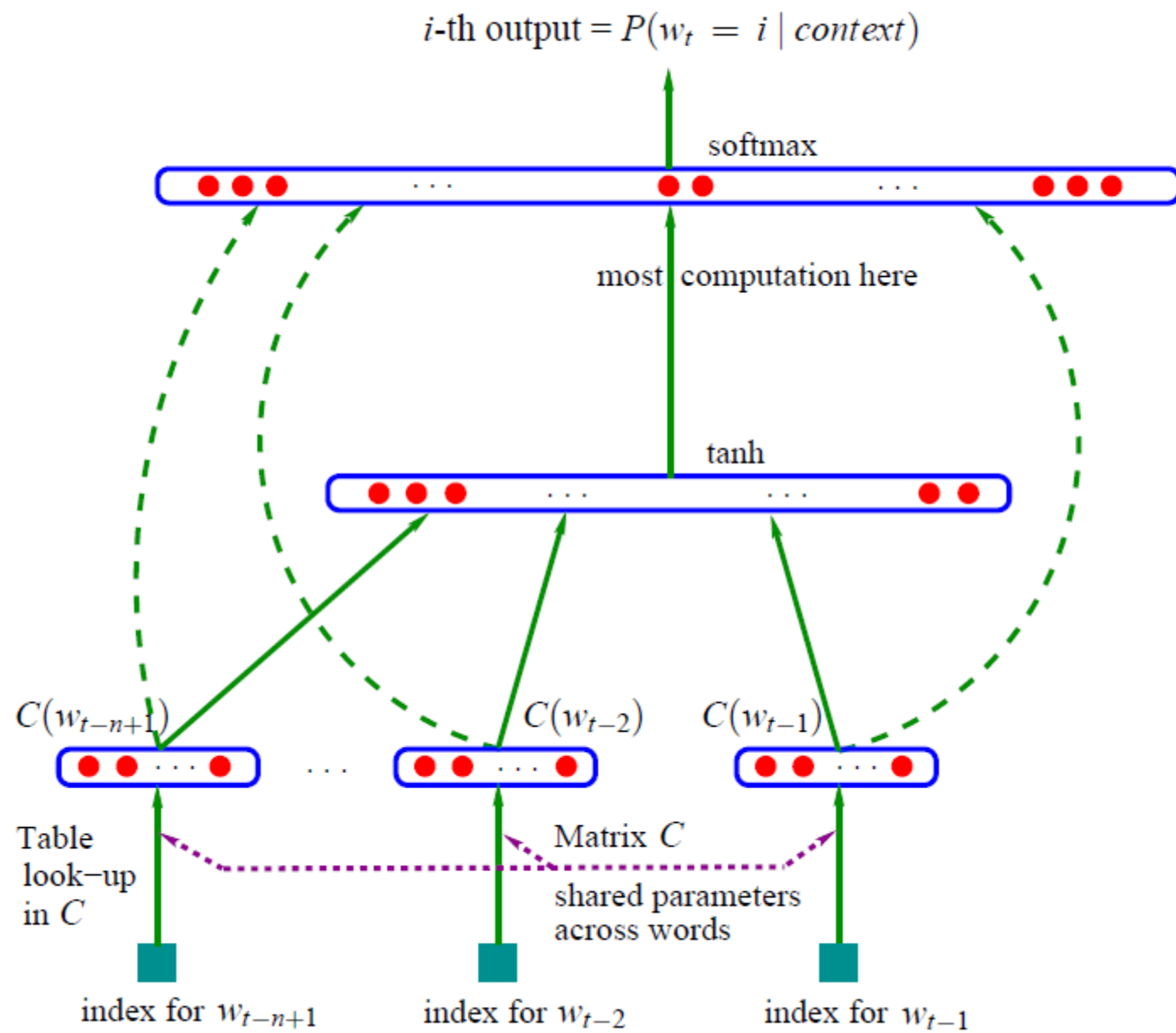
JAUVINC@IRO.UMONTREAL.CA

Editors: Jaz Kandola, Thomas Hofmann, Tomaso Poggio and John Shawe-Taylor

Abstract

A goal of statistical language modeling is to learn the joint probability function of sequences of words in a language. This is intrinsically difficult because of the **curse of dimensionality**: a word sequence on which the model will be tested is likely to be different from all the word sequences seen during training. Traditional but very successful approaches based on n-grams obtain generalization by concatenating very short overlapping sequences seen in the training set. We propose to fight the curse of dimensionality by **learning a distributed representation for words** which allows each training sentence to inform the model about an exponential number of semantically neighboring sentences. The model learns simultaneously (1) a distributed representation for each word along with (2) the probability function for word sequences, expressed in terms of these representations.

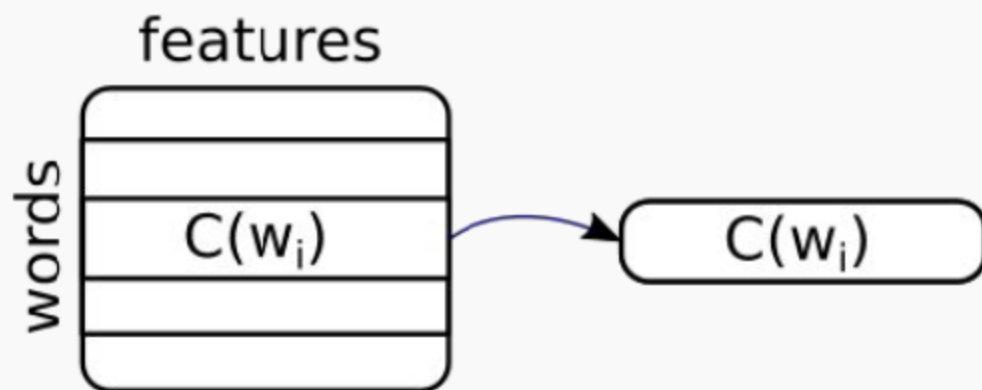
http://web3.cs.columbia.edu/~blei/seminar/2016_discrete_data/readings/BengioDucharmeVincentJanvin2003.pdf



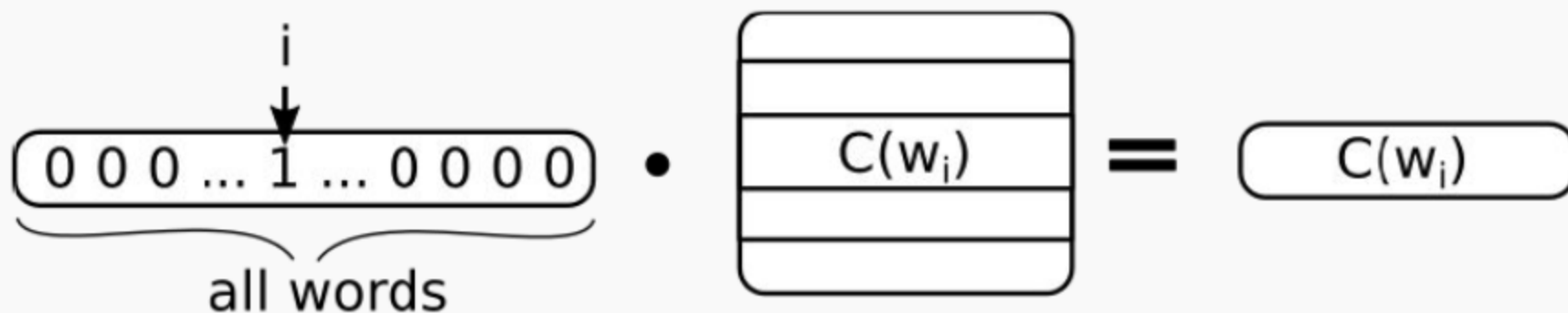
Embedding matrix

Two ways of thinking about the embedding matrix.

1. **Each row** contains a word embedding, which we need to extract



2. It is a normal **weight matrix**, working with a 1-hot input vector



WORD REPRESENTATIONS

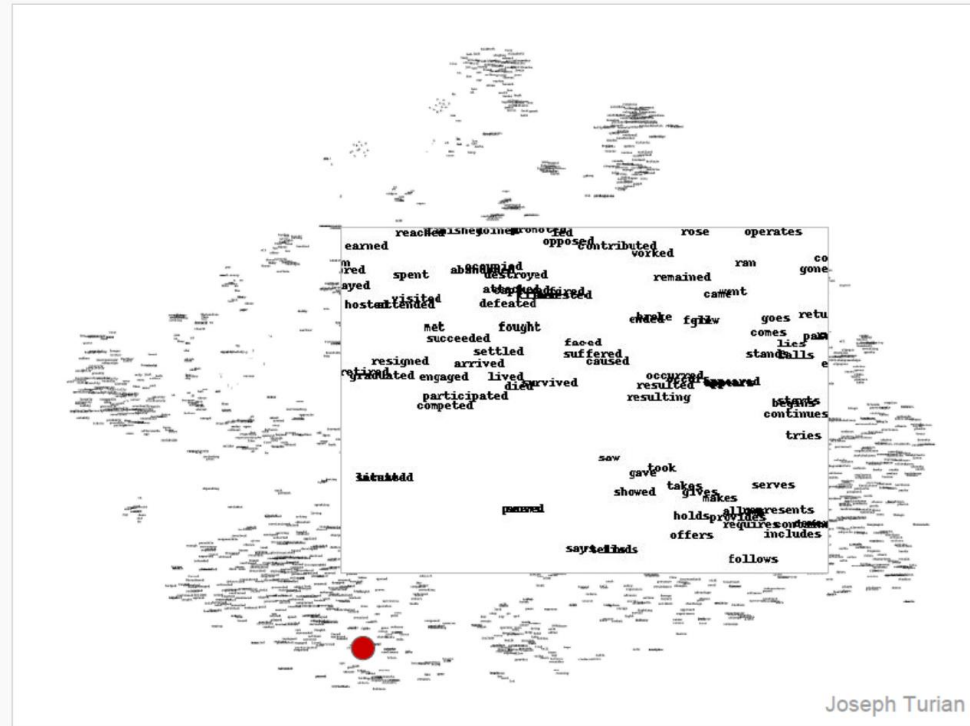
Topics: continuous word representation

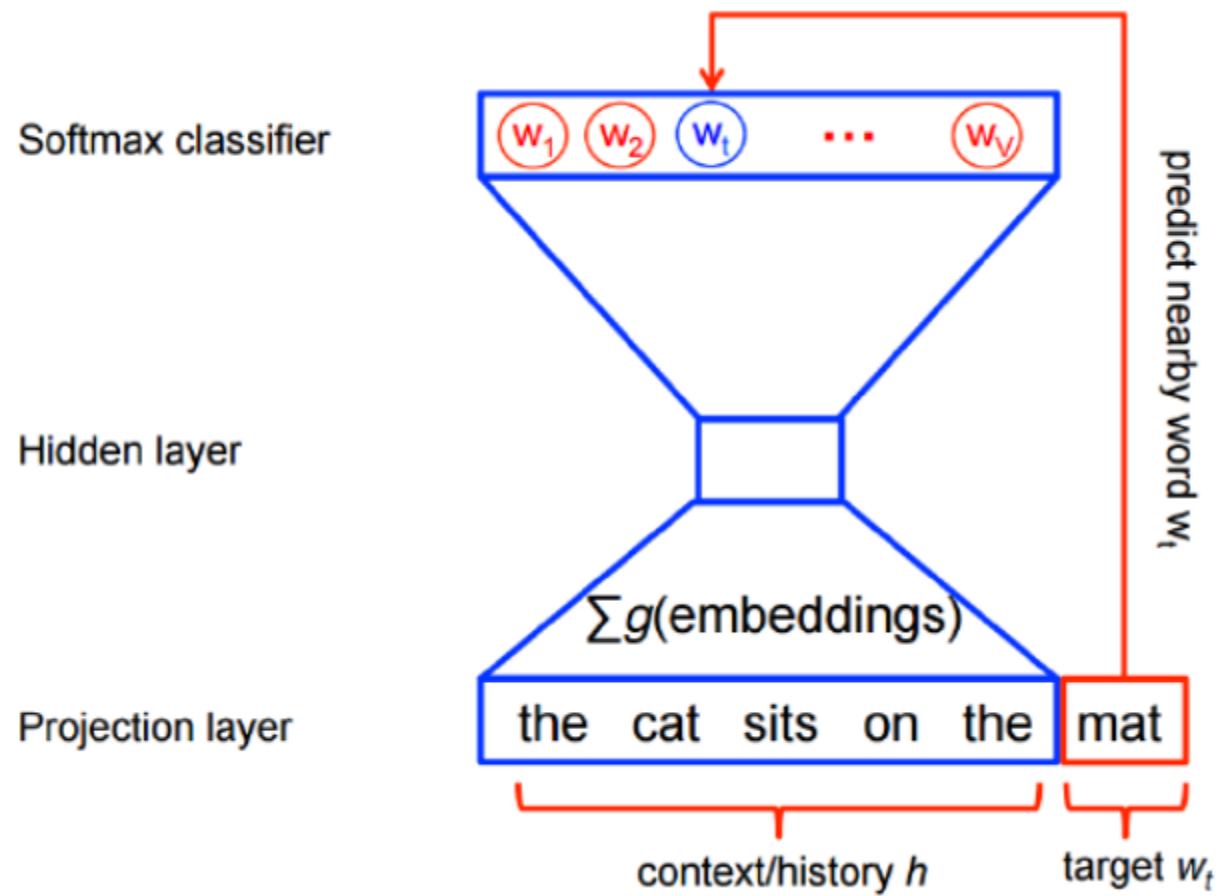
- Idea: learn a continuous representation of words

- each word w is associated with a real-valued vector $C(w)$

Word	w	$C(w)$
" the "	1	[0.6762, -0.9607, 0.3626, -0.2410, 0.6636]
" a "	2	[0.6859, -0.9266, 0.3777, -0.2140, 0.6711]
" have "	3	[0.1656, -0.1530, 0.0310, -0.3321, -0.1342]
" be "	4	[0.1760, -0.1340, 0.0702, -0.2981, -0.1111]
" cat "	5	[0.5896, 0.9137, 0.0452, 0.7603, -0.6541]
" dog "	6	[0.5965, 0.9143, 0.0899, 0.7702, -0.6392]
" car "	7	[-0.0069, 0.7995, 0.6433, 0.2898, 0.6359]
...	...	...

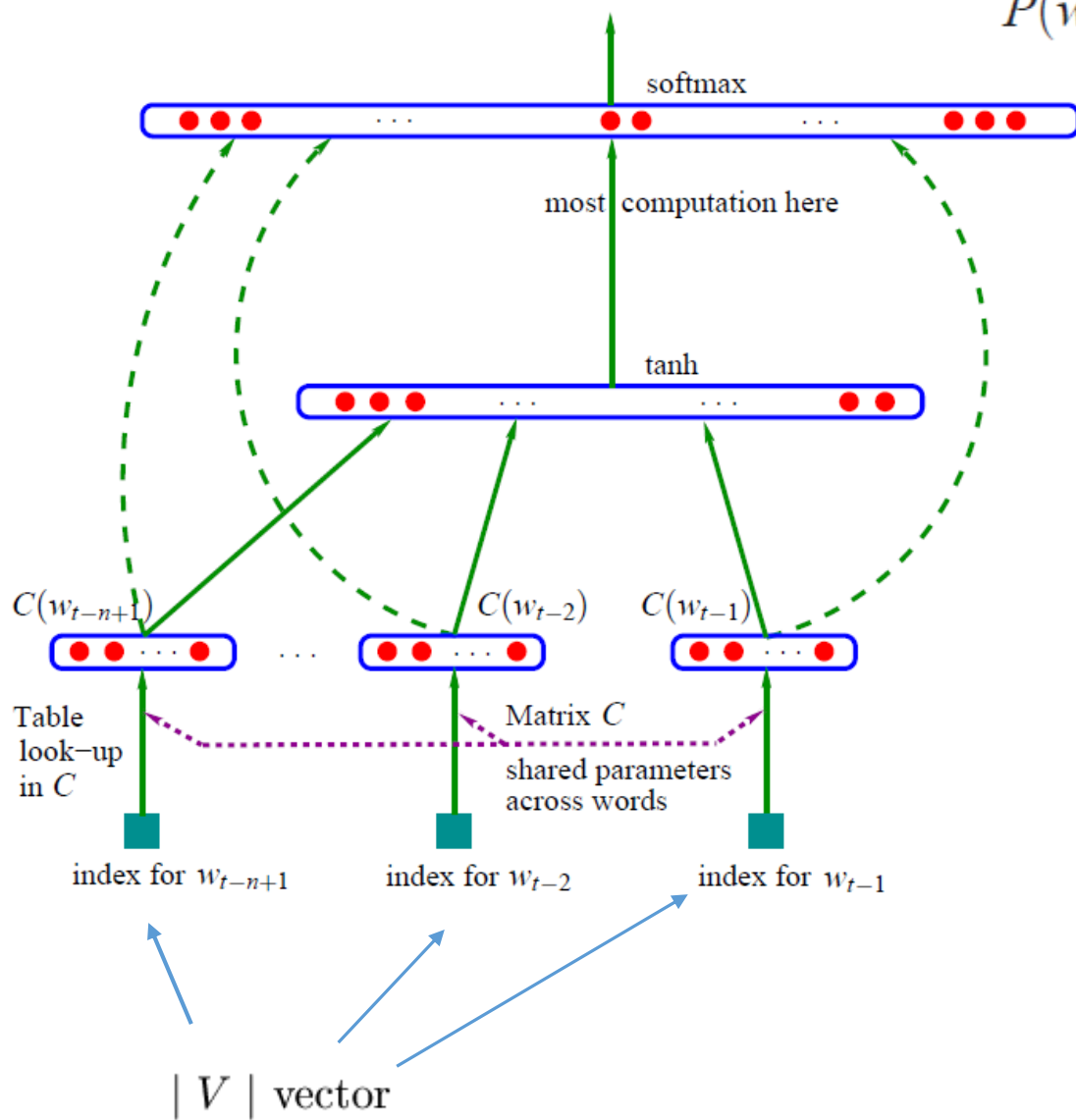
Word similarity

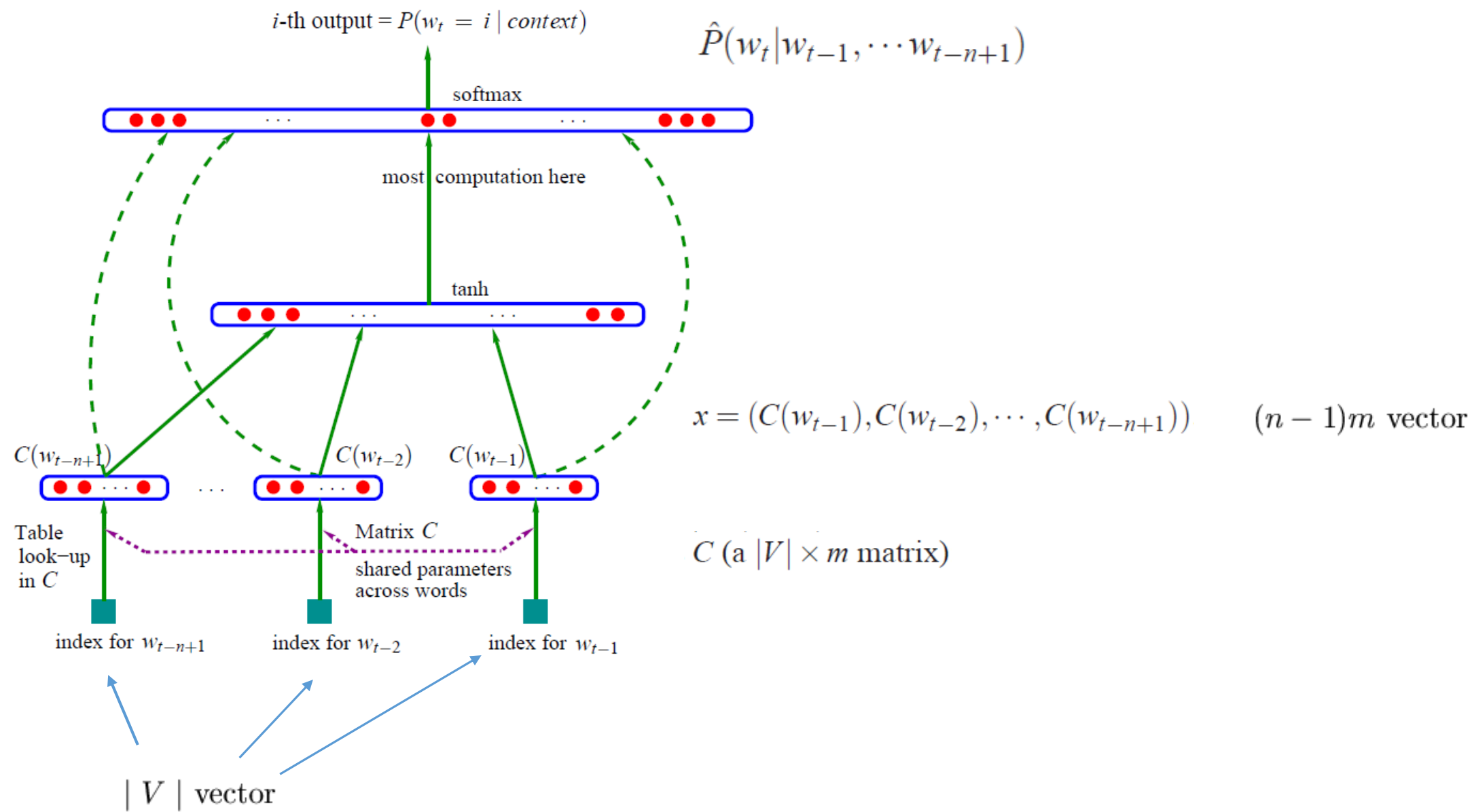


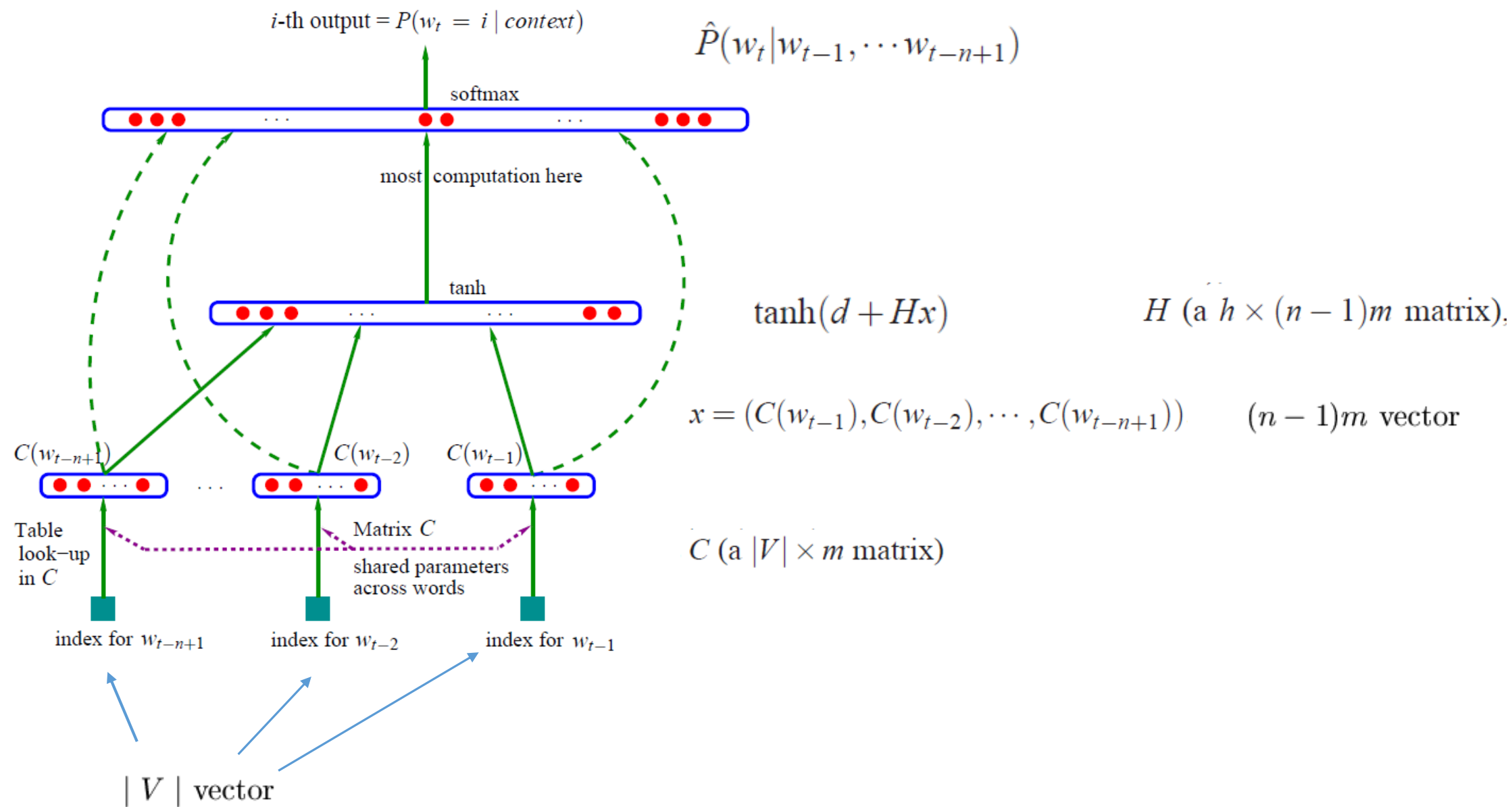


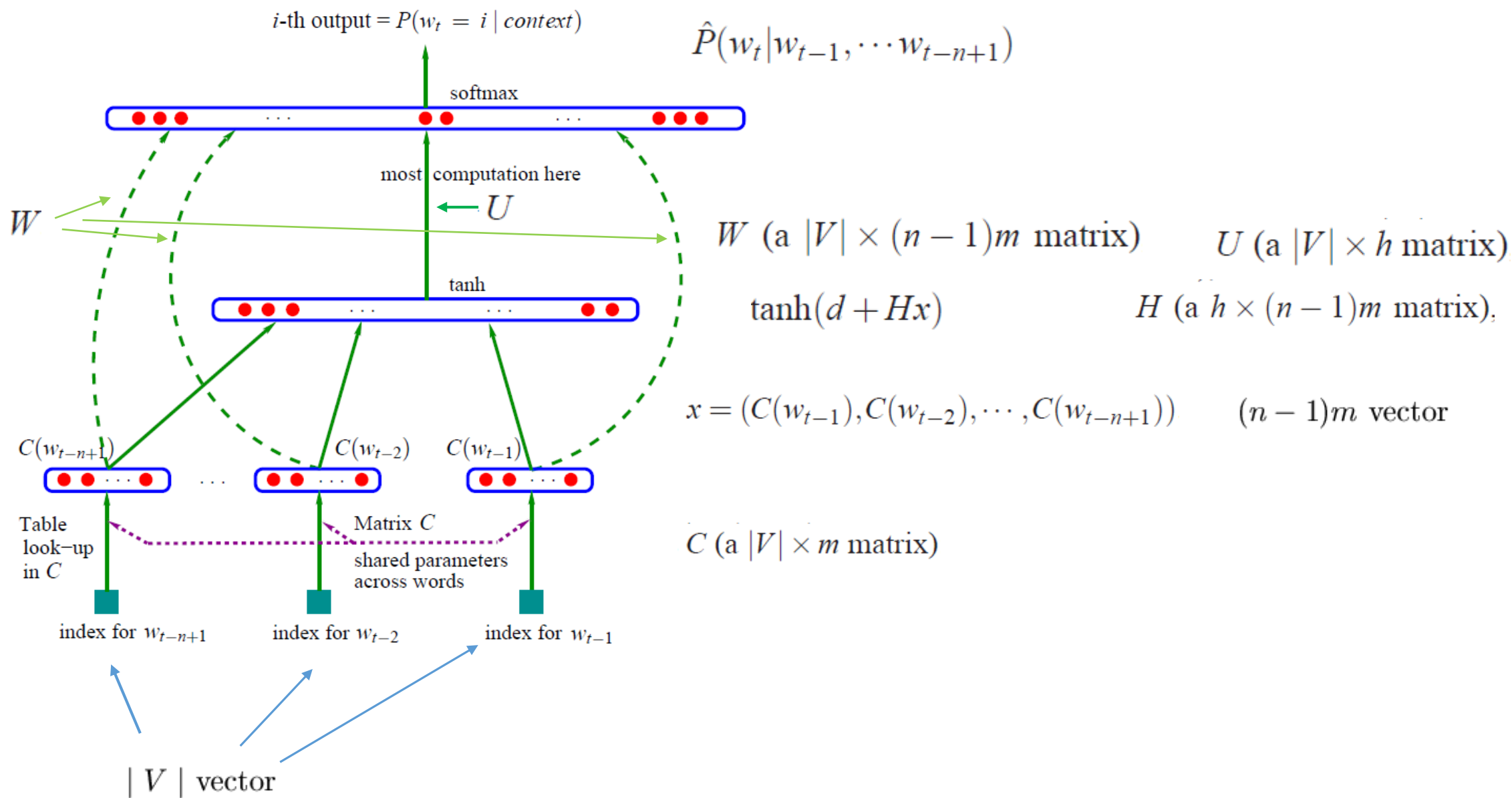
i -th output = $P(w_t = i \mid \text{context})$

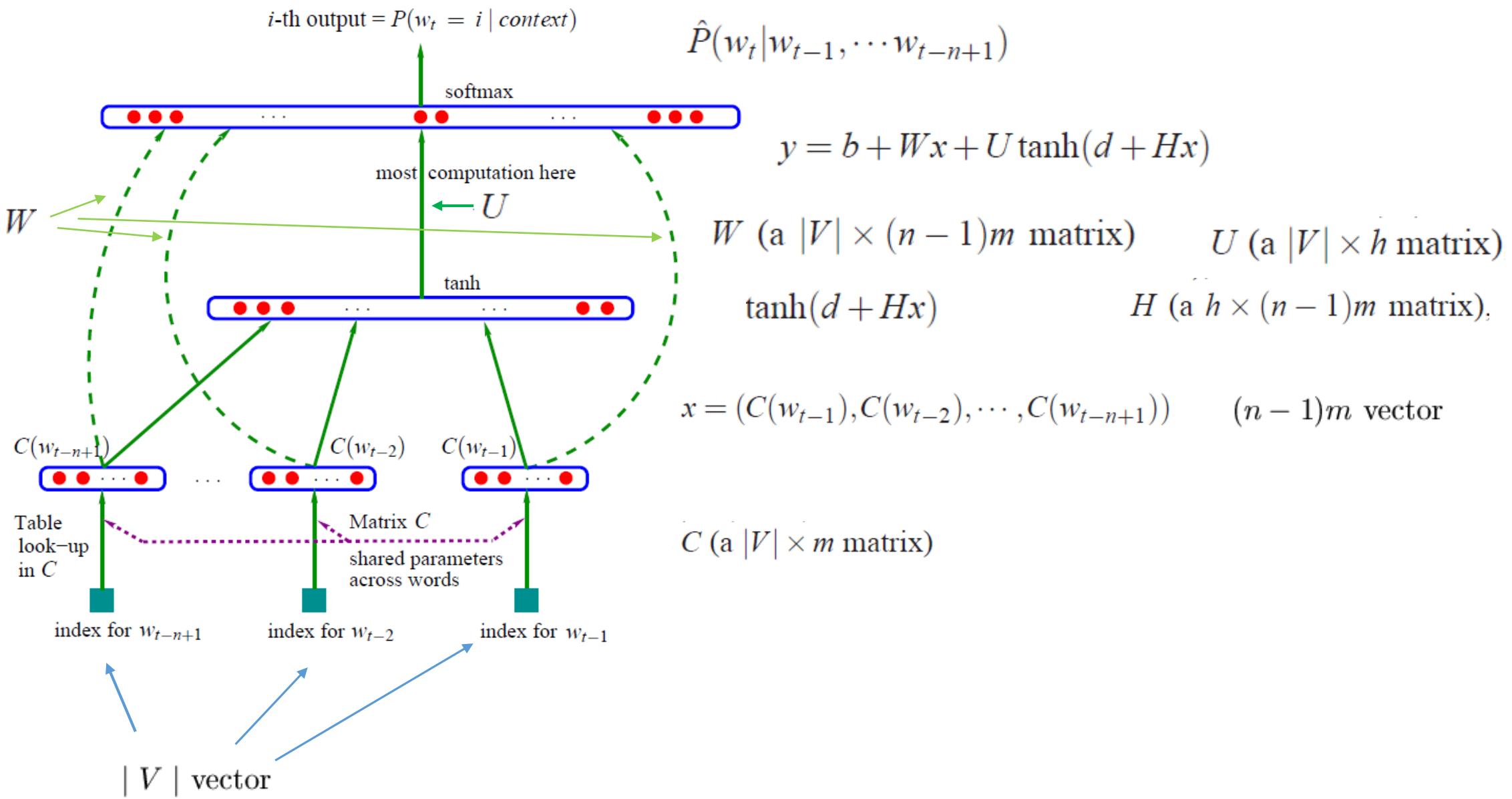
$$\hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1})$$

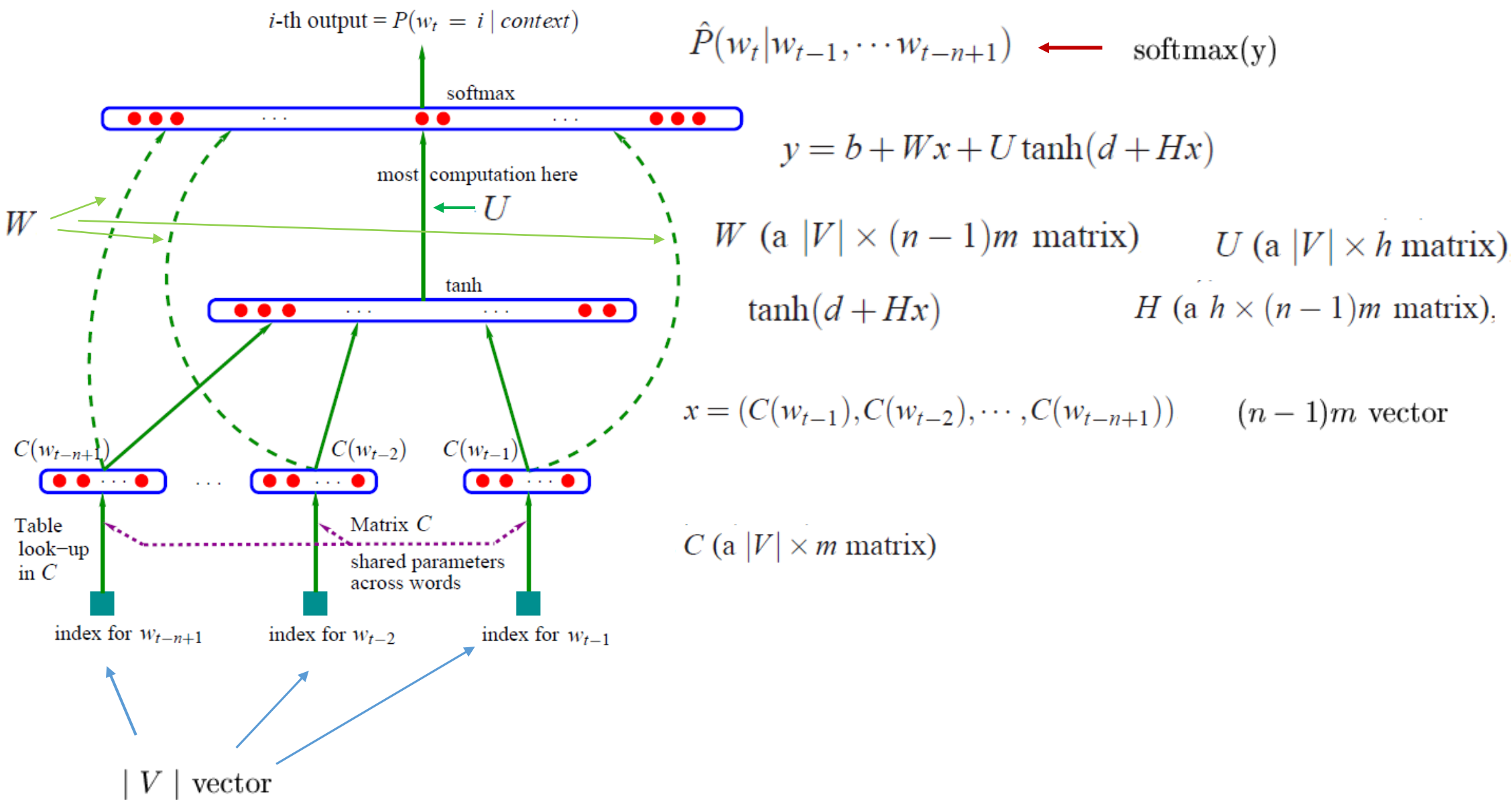


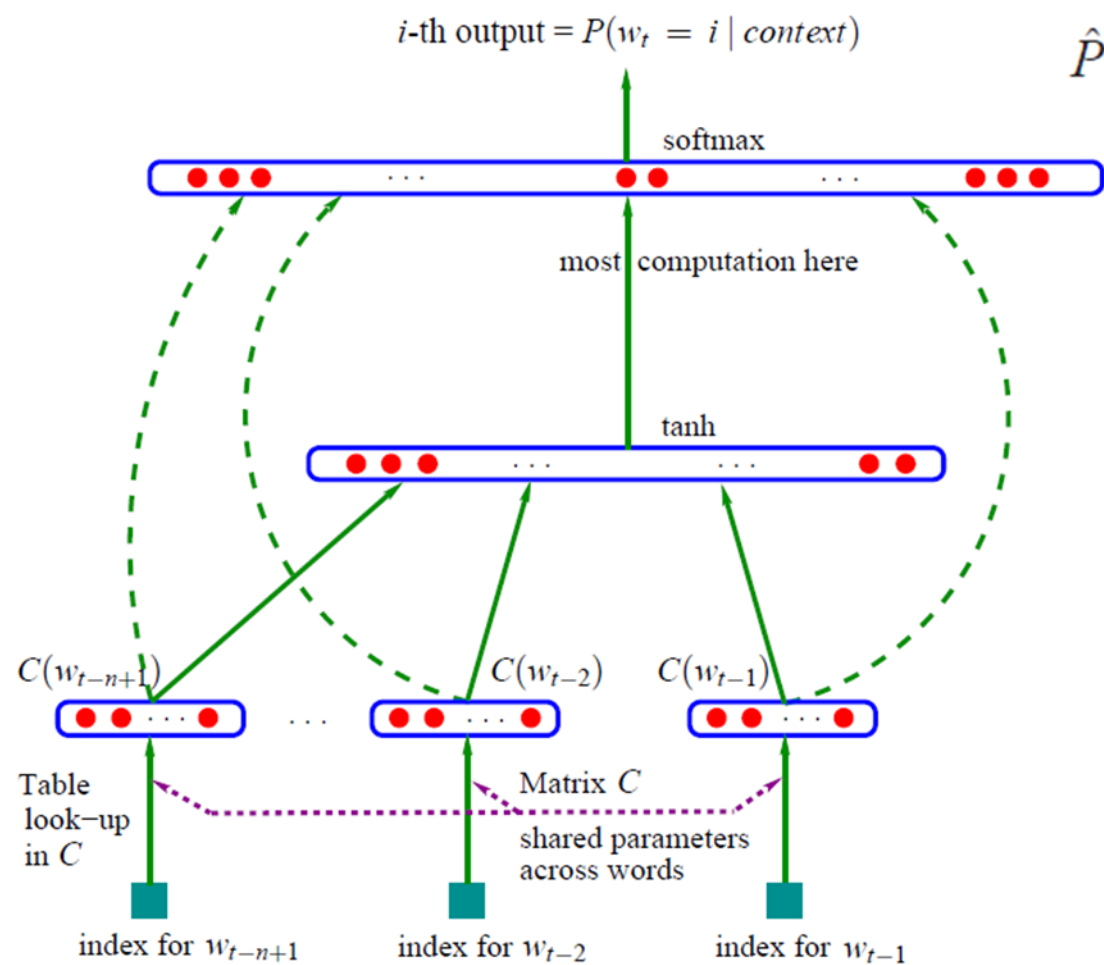






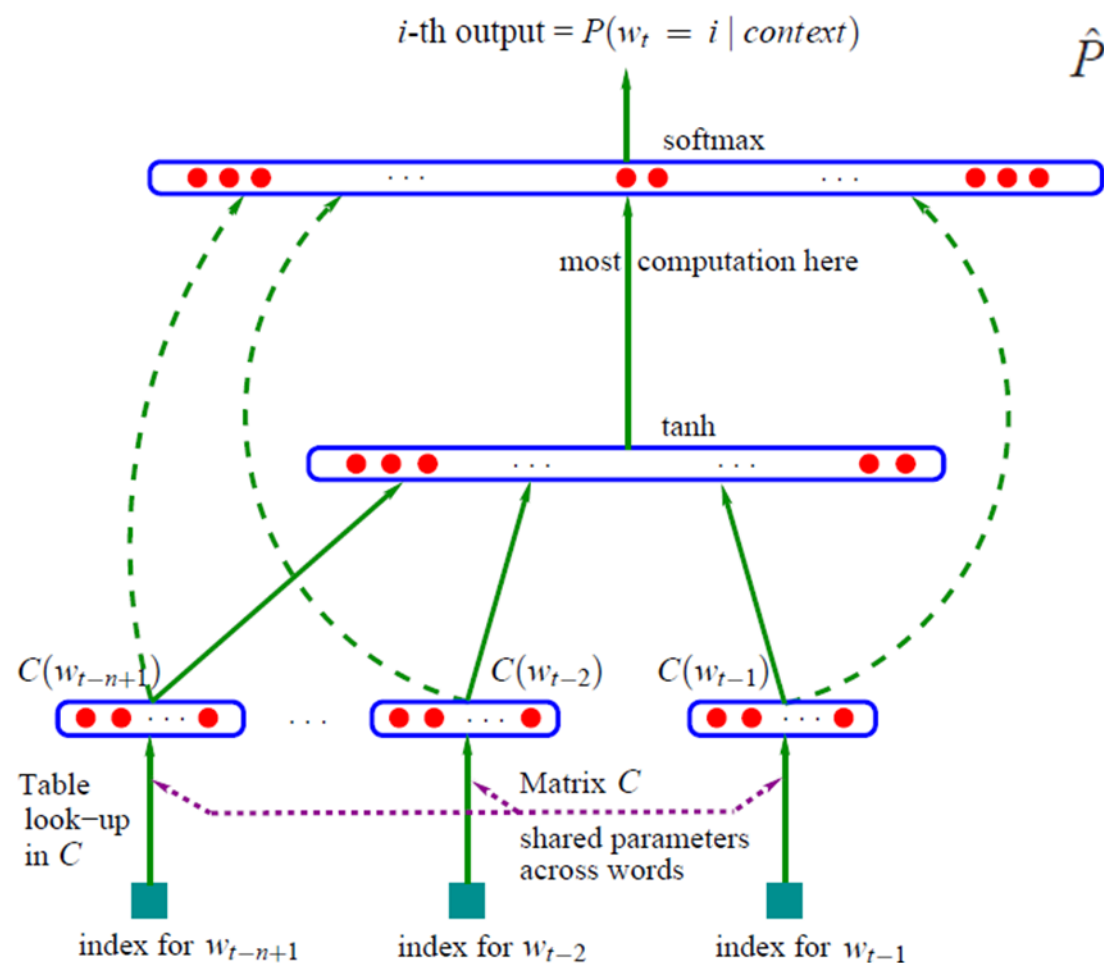






$$\hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1})$$

$$\theta = (b, d, W, U, H, C).$$



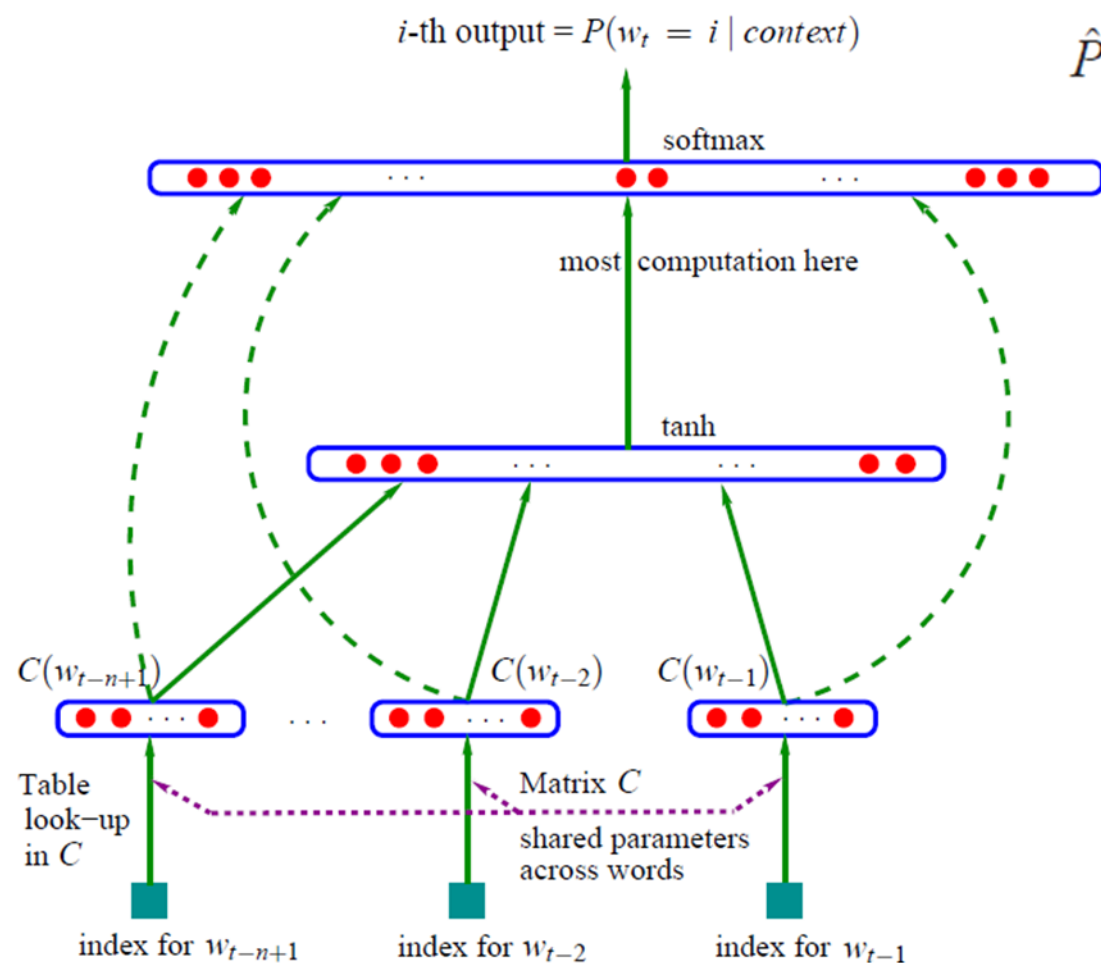
$$\hat{P}(w_t | w_{t-1}, \dots, w_{t-n+1})$$

$$\theta = (b, d, W, U, H, C).$$



The number of free parameters is

$$|V|(1 + nm + h) + h(1 + (n - 1)m)$$



$$\hat{P}(w_t | w_{t-1}, \dots w_{t-n+1})$$

$$\theta = (b, d, W, U, H, C).$$



The number of free parameters is

$$|V|(1 + nm + h) + h(1 + (n - 1)m)$$

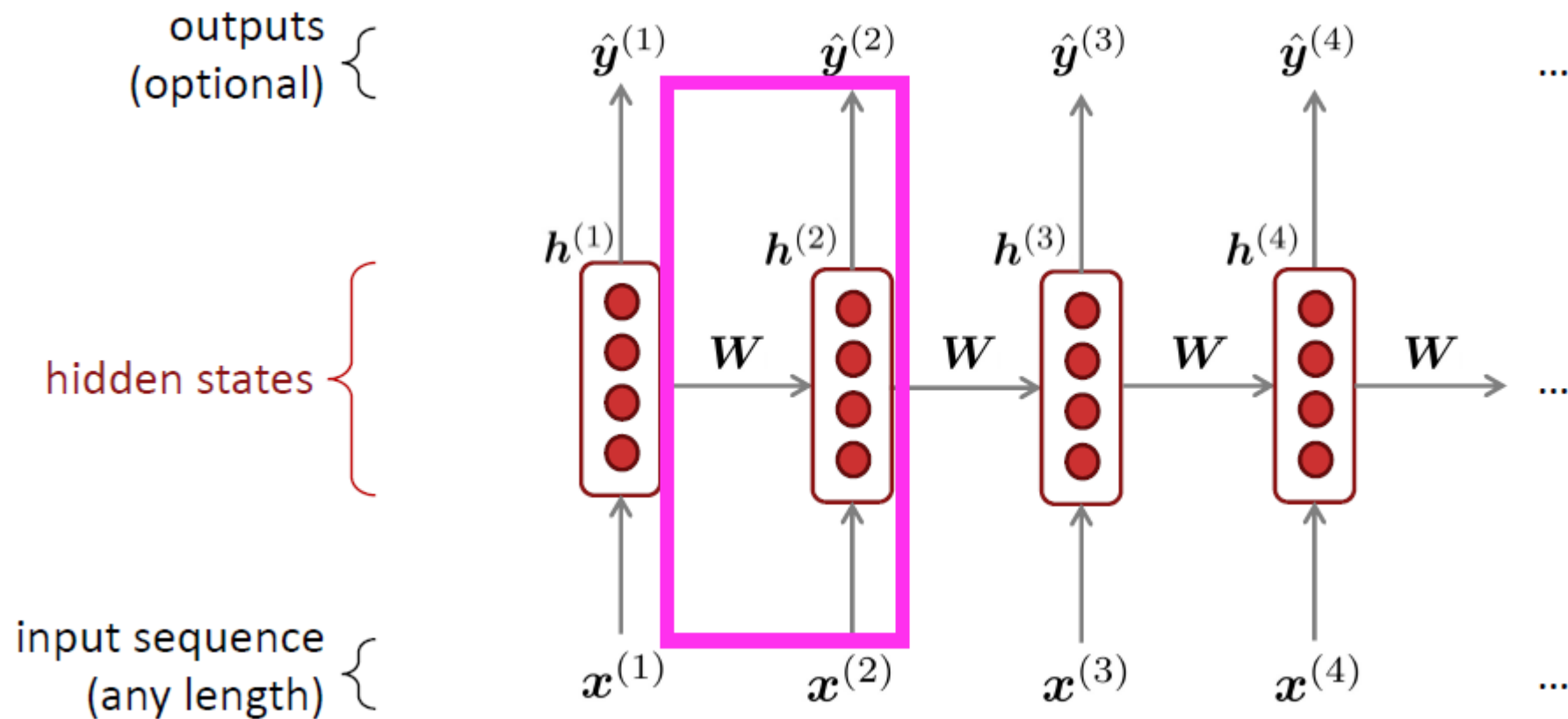
$$\text{cost} = -\log \hat{P}(w_t | w_{t-1}, \dots w_{t-n+1})$$



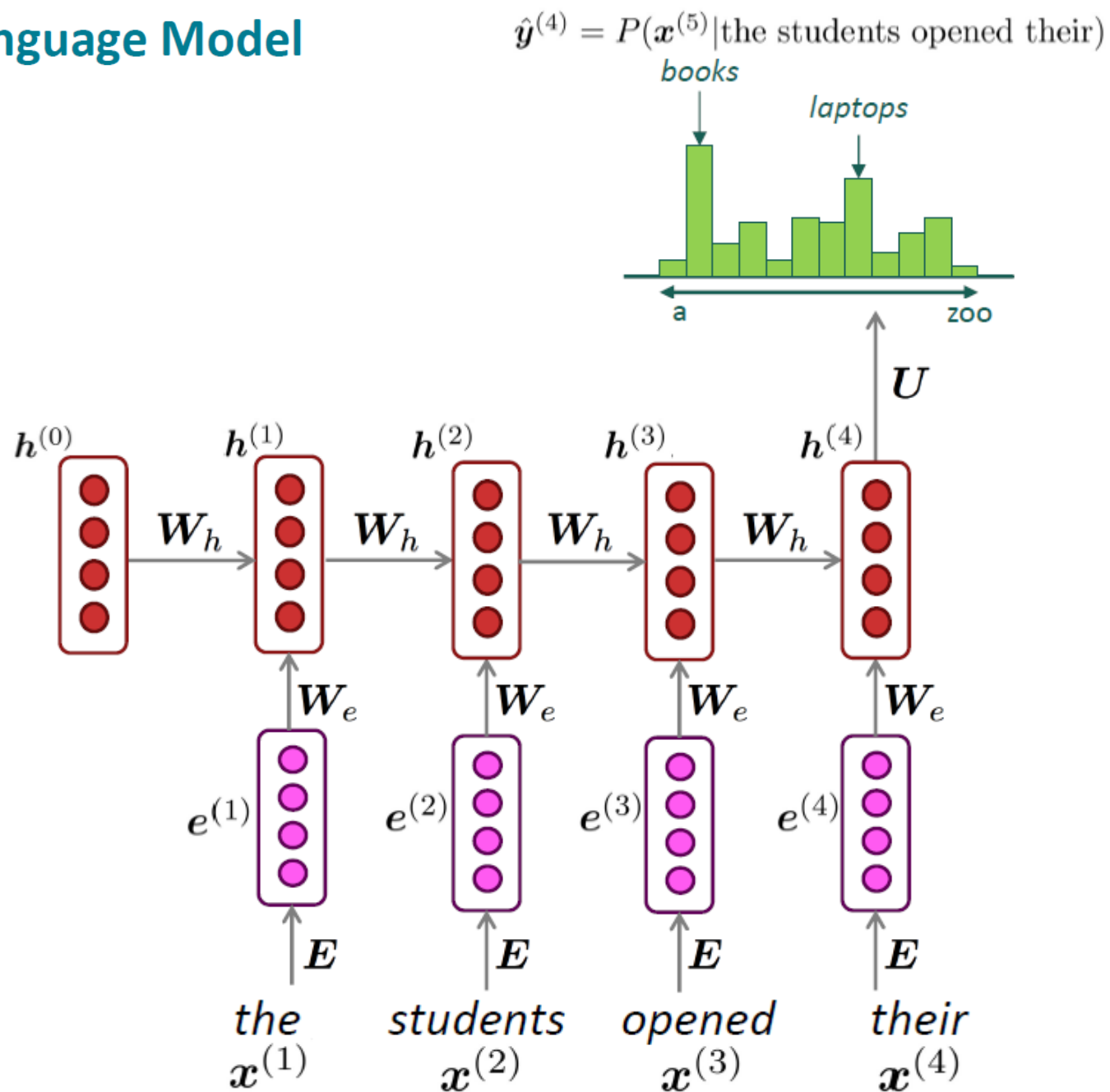
$$\theta \leftarrow \theta + \varepsilon \frac{\partial \log \hat{P}(w_t | w_{t-1}, \dots w_{t-n+1})}{\partial \theta}$$

3. Recurrent Neural Networks (RNN)

A family of neural architectures

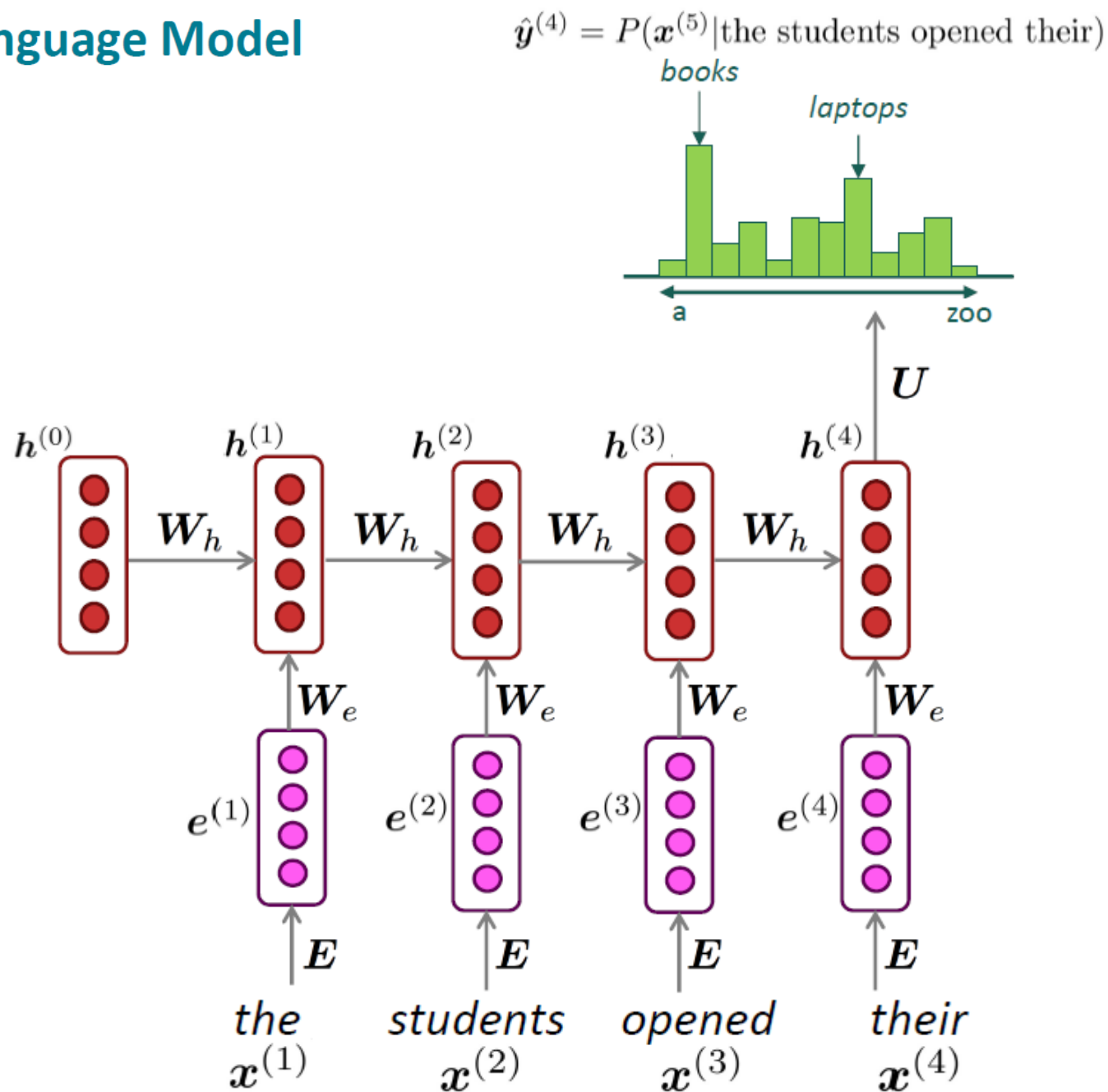


A Simple RNN Language Model



A Simple RNN Language Model

words / one-hot vectors
 $\mathbf{x}^{(t)} \in \mathbb{R}^{|V|}$



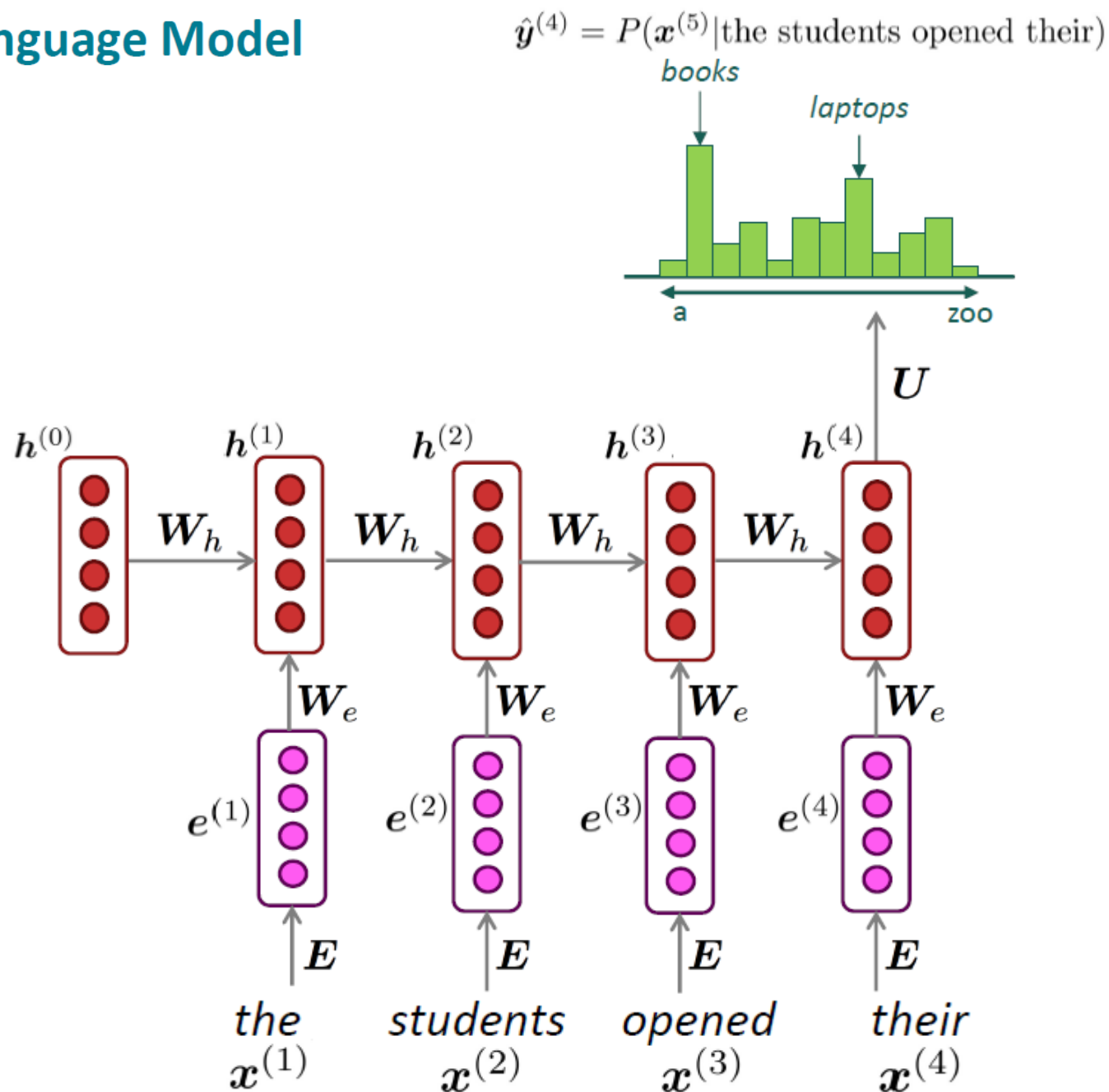
A Simple RNN Language Model

word embeddings

$$e^{(t)} = Ex^{(t)}$$

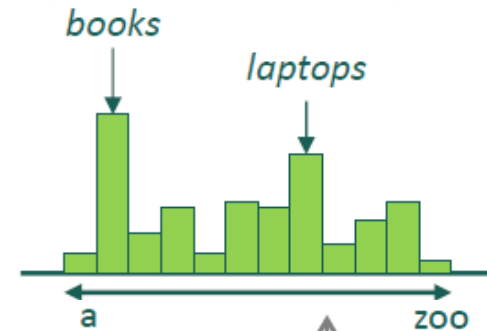
words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



A Simple RNN Language Model

$$\hat{y}^{(4)} = P(x^{(5)} | \text{the students opened their})$$



hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

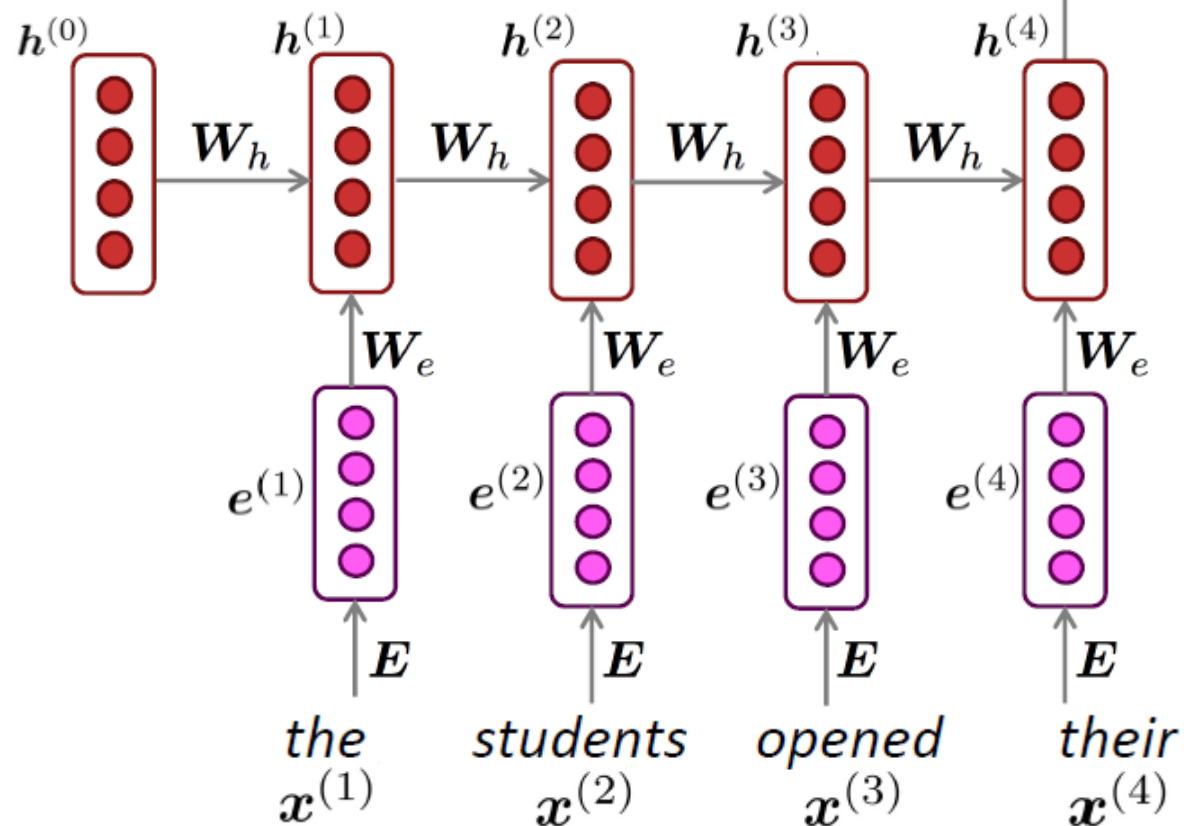
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



A Simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax} \left(U h^{(t)} + b_2 \right) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma \left(W_h h^{(t-1)} + W_e e^{(t)} + b_1 \right)$$

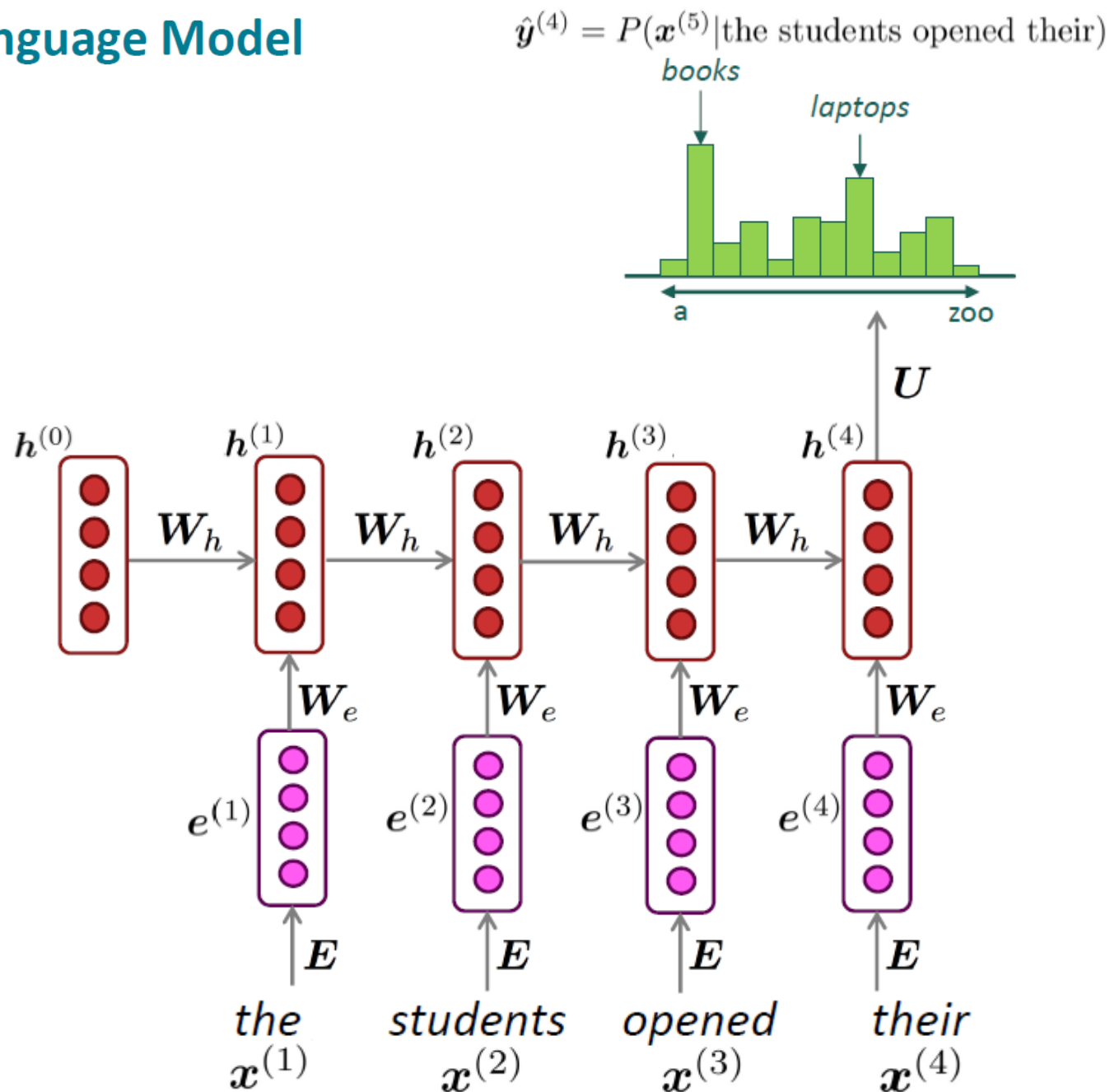
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



A Simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax} \left(U h^{(t)} + b_2 \right) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma \left(W_h h^{(t-1)} + W_e e^{(t)} + b_1 \right)$$

$h^{(0)}$ is the initial hidden state

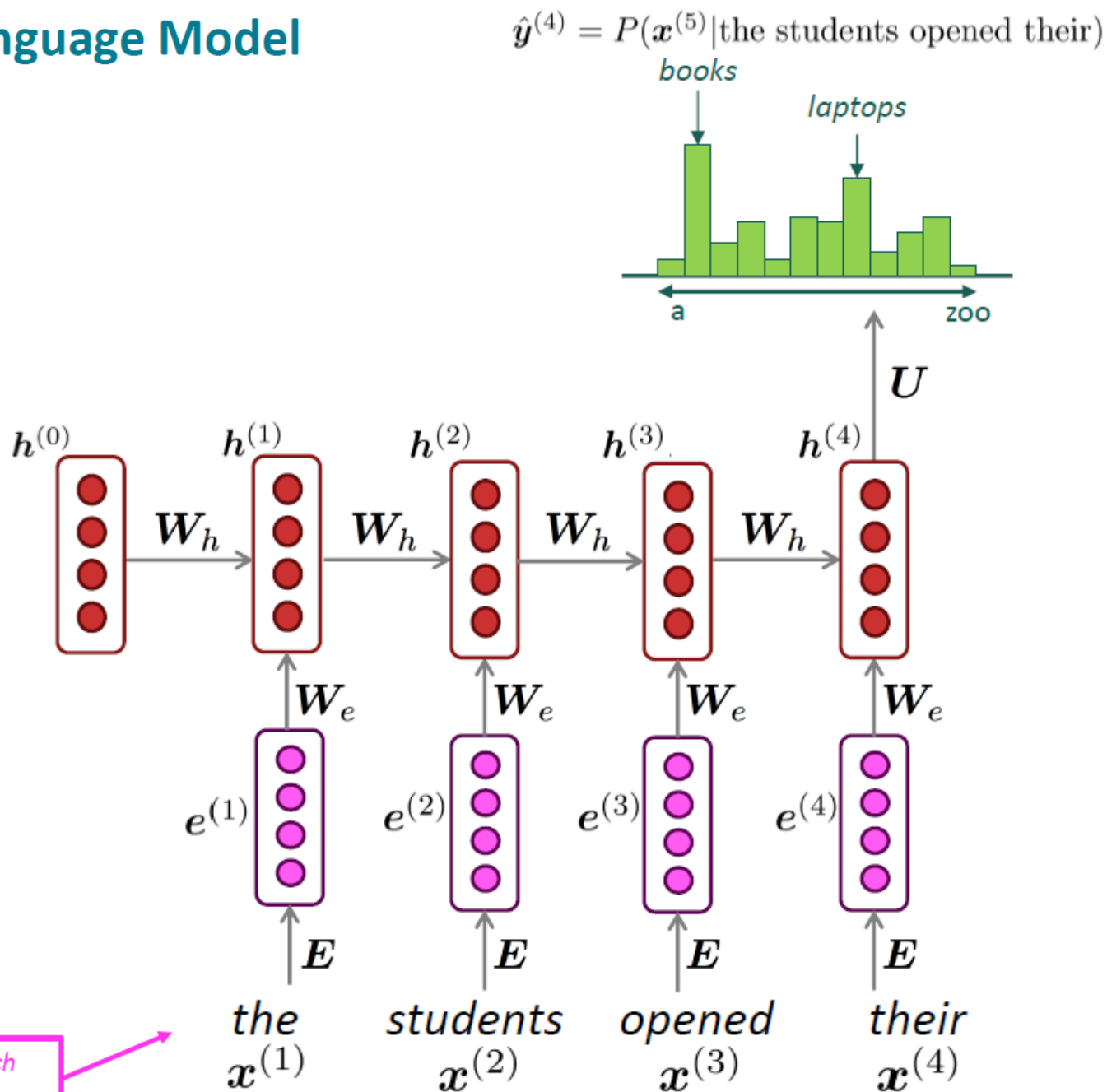
word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$

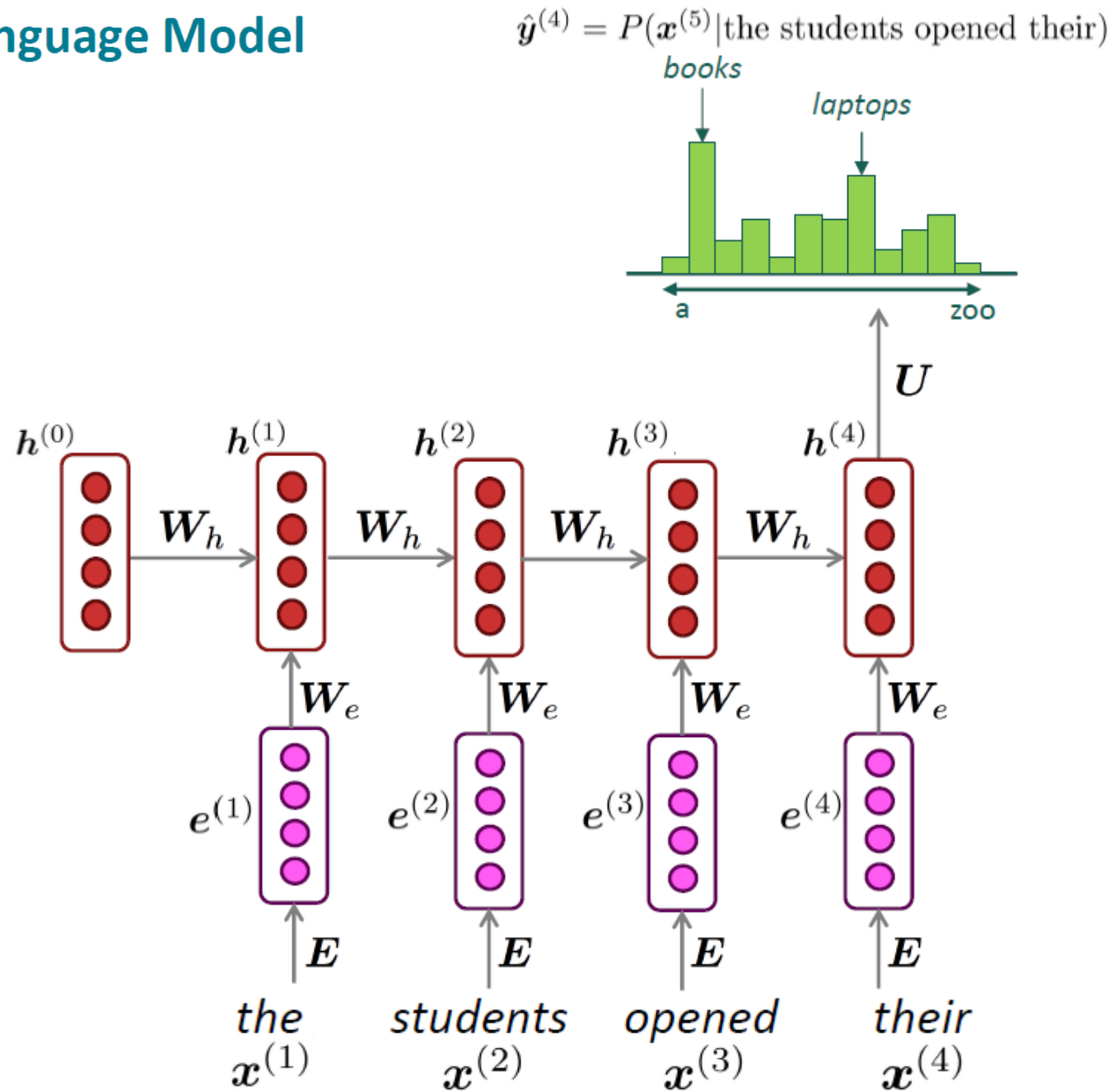
Note: this input sequence could be much longer now!



A Simple RNN Language Model

RNN Advantages:

- Can process **any length** input
- Computation for step t can (in theory) use information from **many steps back**
- **Model size doesn't increase** for longer input context
- Same weights applied on every timestep, so there is **symmetry** in how inputs are processed.



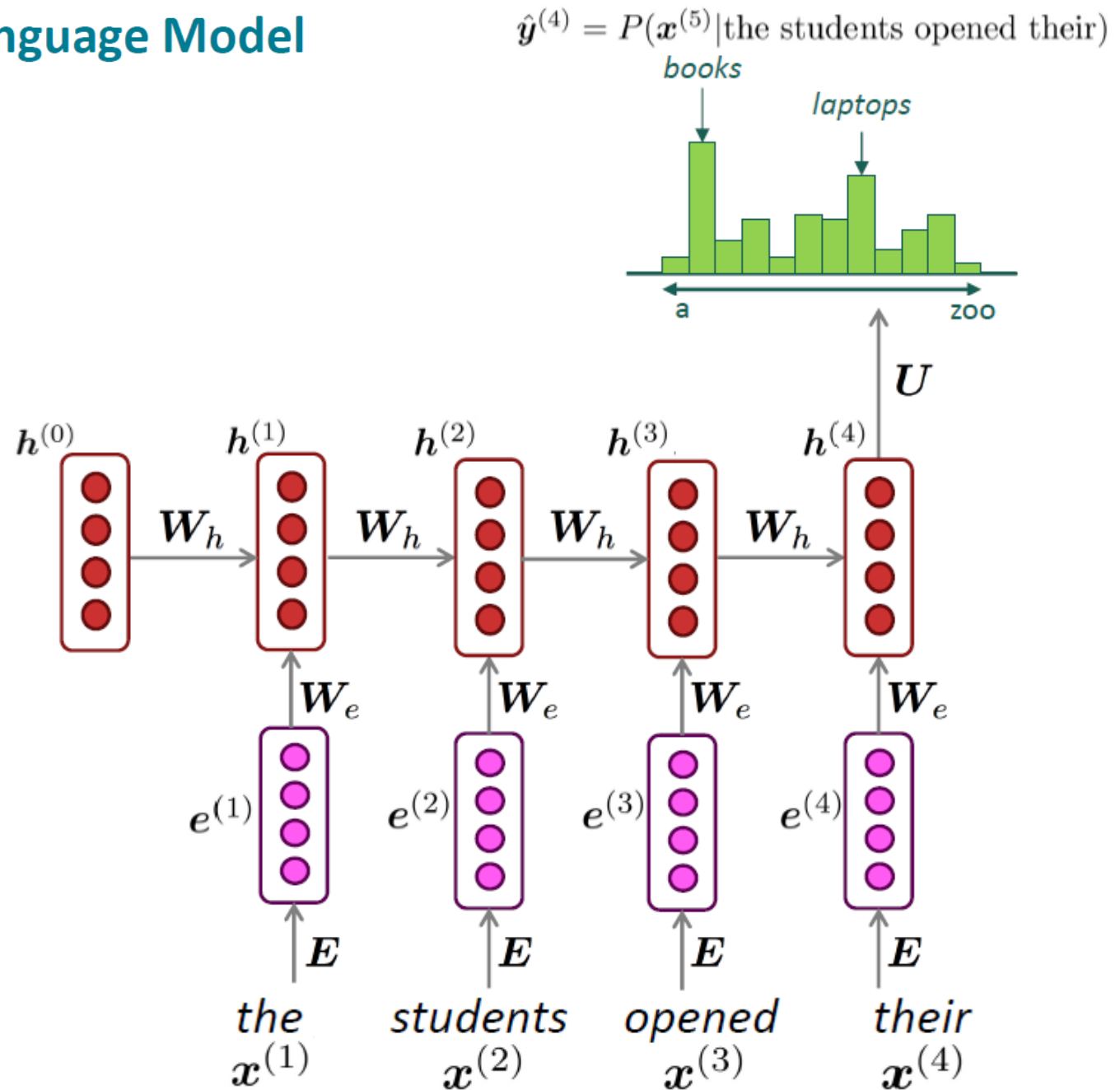
A Simple RNN Language Model

RNN Advantages:

- Can process **any length** input
- Computation for step t can (in theory) use information from **many steps back**
- **Model size doesn't increase** for longer input context
- Same weights applied on every timestep, so there is **symmetry** in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is **slow**
- In practice, difficult to access information from **many steps back**



Training an RNN Language Model

Training an RNN Language Model

- Get a **big corpus of text** which is a sequence of words $x^{(1)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ **for every step t** .
 - i.e., predict probability dist of *every word*, given words so far

Training an RNN Language Model

- Get a **big corpus of text** which is a sequence of words $x^{(1)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{\mathbf{y}}^{(t)}$ **for every step t** .
 - i.e., predict probability dist of *every word*, given words so far
- **Loss function** on step t is **cross-entropy** between predicted probability distribution $\hat{\mathbf{y}}^{(t)}$, and the true next word $\mathbf{y}^{(t)}$ (one-hot for $x^{(t+1)}$):

$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{x_{t+1}}^{(t)}$$

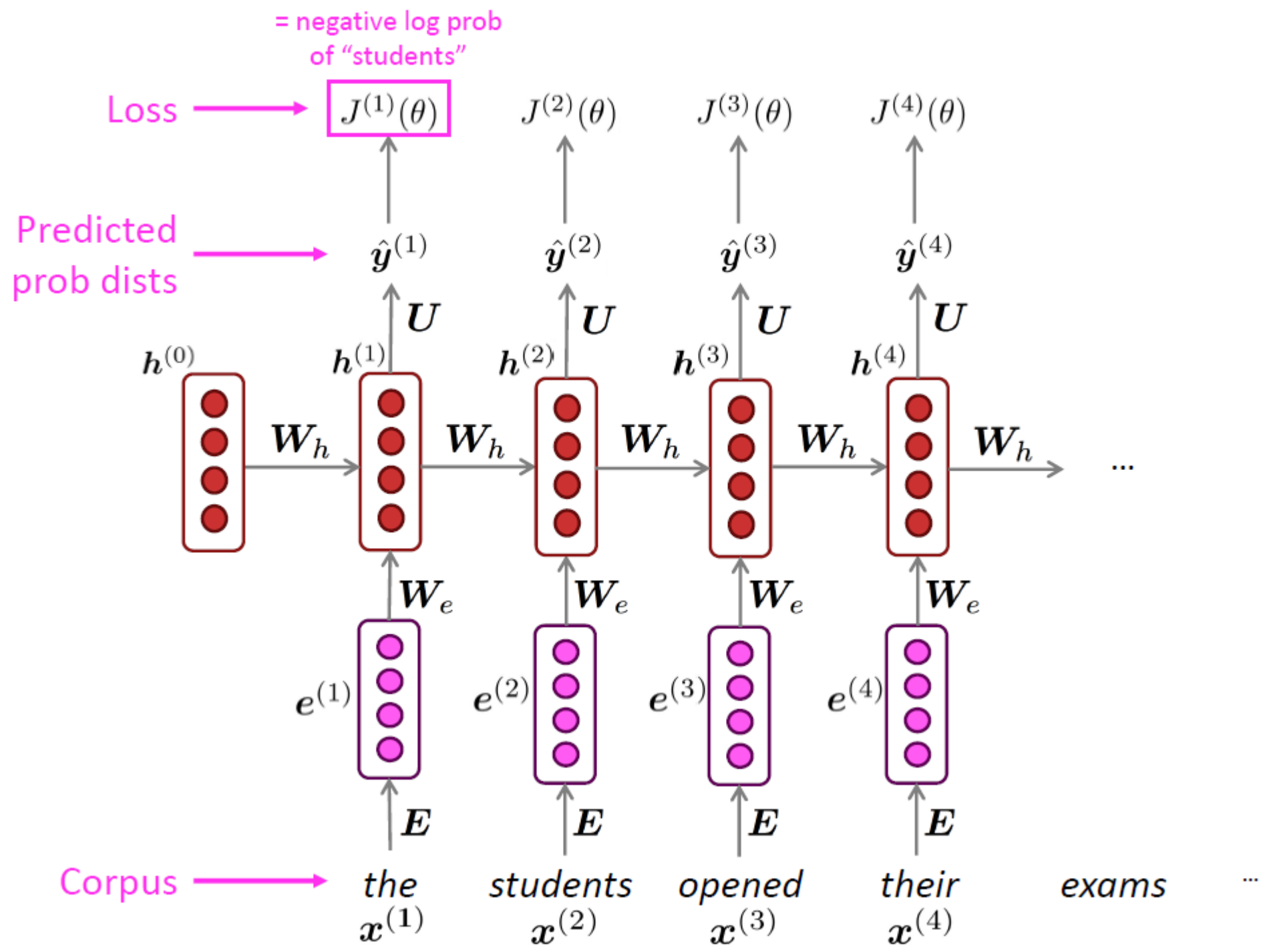
Training an RNN Language Model

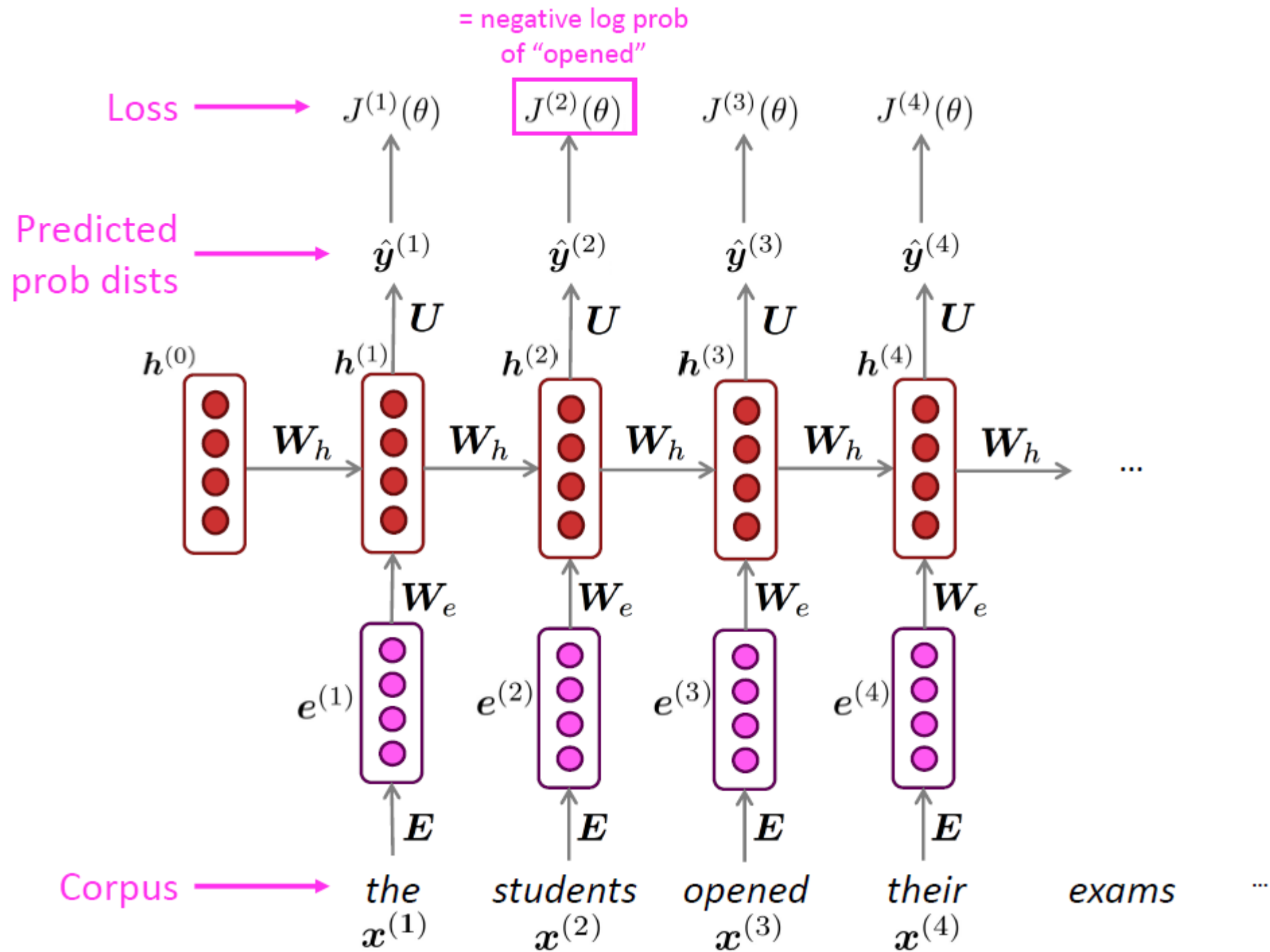
- Get a **big corpus of text** which is a sequence of words $x^{(1)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{\mathbf{y}}^{(t)}$ **for every step t** .
 - i.e., predict probability dist of *every word*, given words so far
- **Loss function** on step t is **cross-entropy** between predicted probability distribution $\hat{\mathbf{y}}^{(t)}$, and the true next word $\mathbf{y}^{(t)}$ (one-hot for $x^{(t+1)}$):

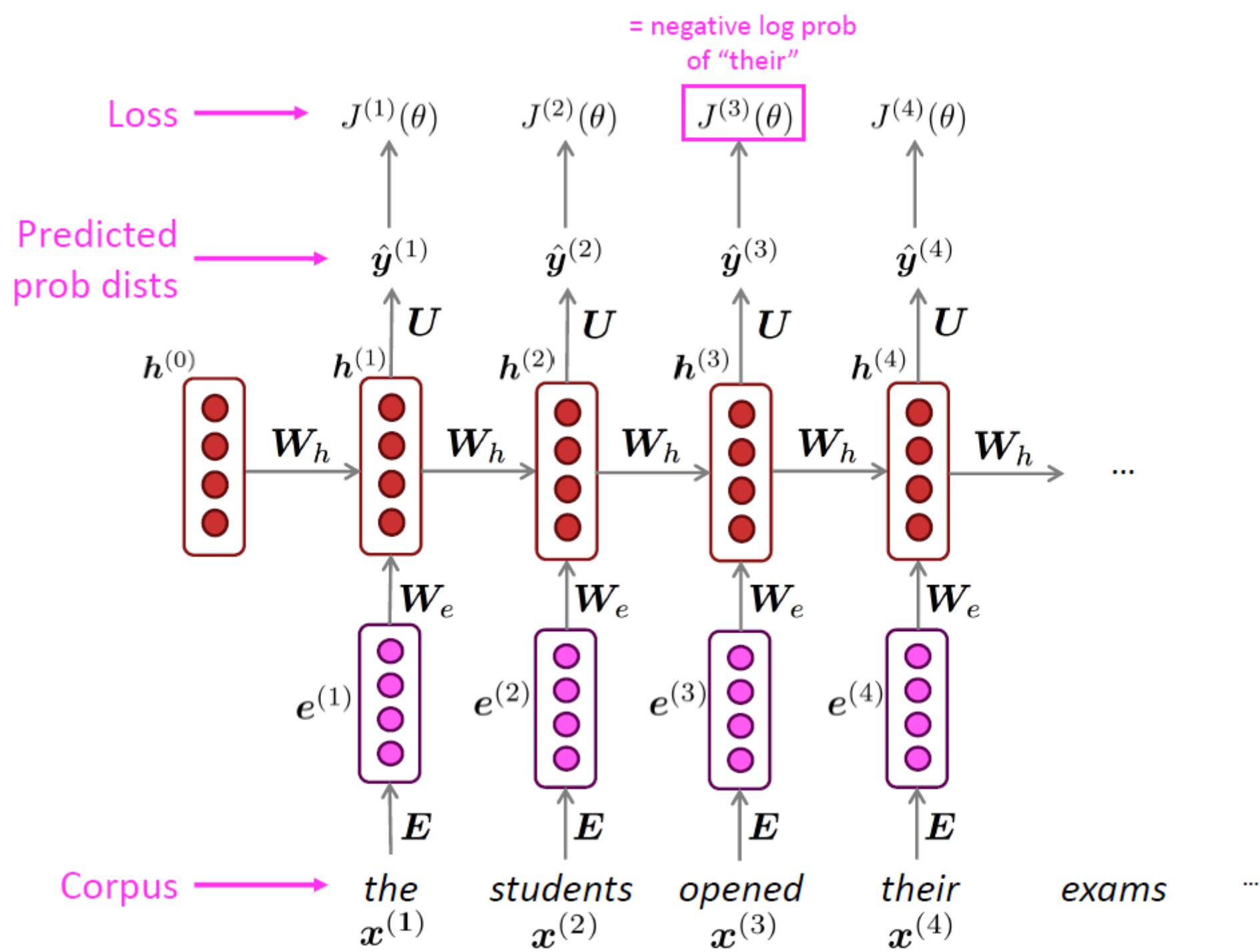
$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$

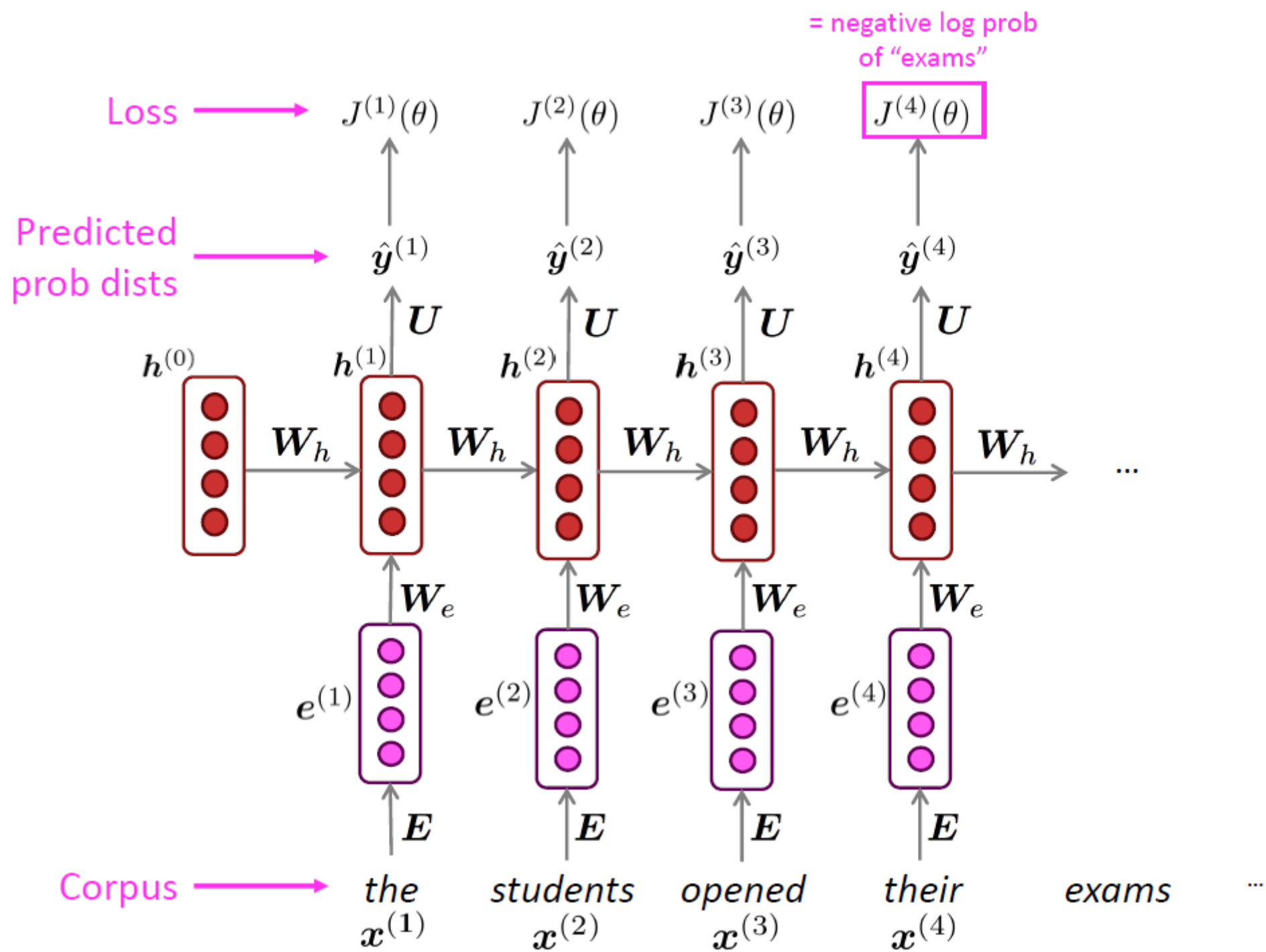
- Average this to get **overall loss** for entire training set:

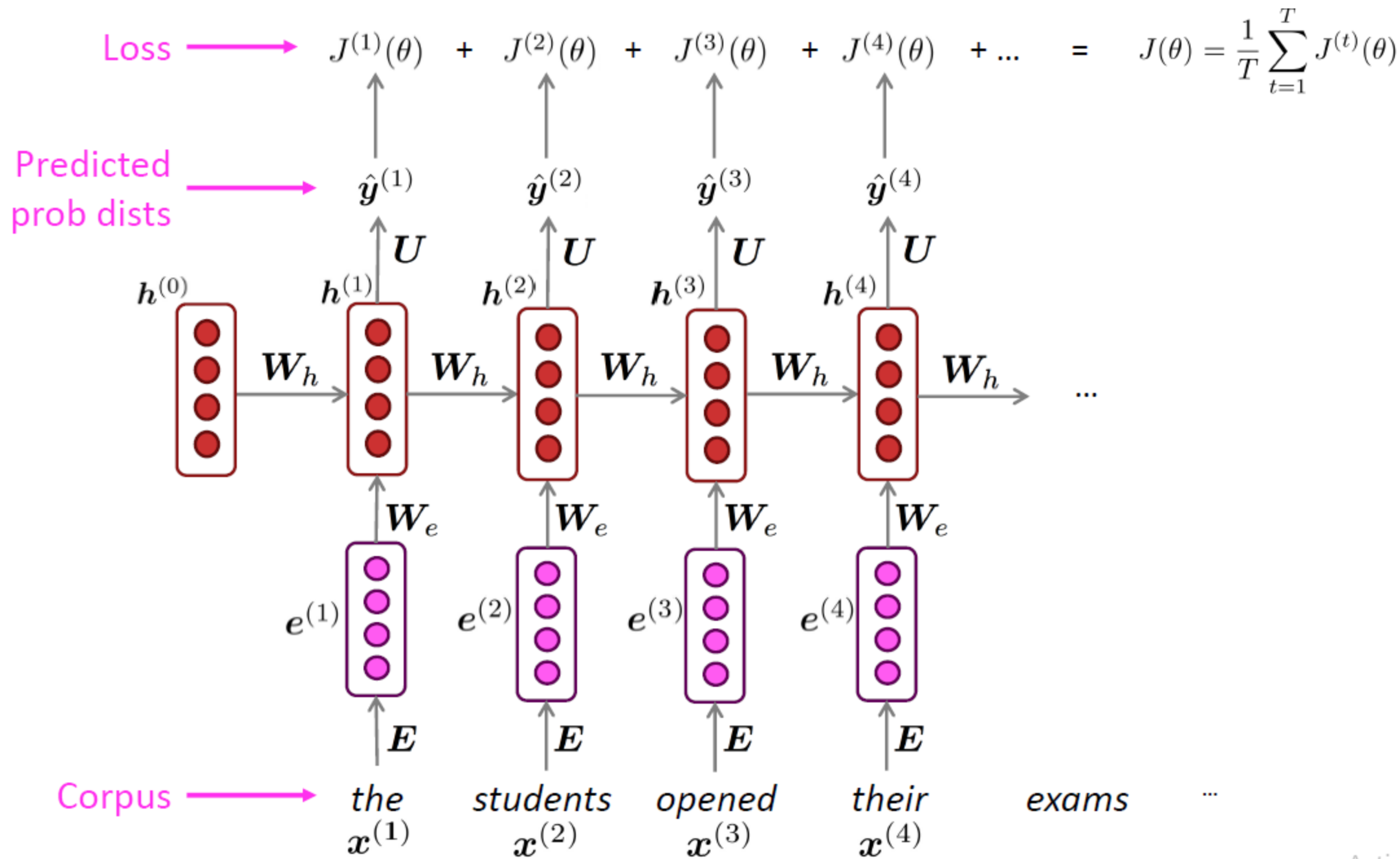
$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$











Training a RNN Language Model

- However: Computing loss and gradients across **entire corpus** $x^{(1)}, \dots, x^{(T)}$ at once is **too expensive** (memory-wise)!

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

- In practice, consider $x^{(1)}, \dots, x^{(T)}$ as a **sentence** (or a **document**)
- Recall: **Stochastic Gradient Descent** allows us to compute loss and gradients for small chunk of data, and update.
- Compute loss $J(\theta)$ for a sentence (actually, a batch of sentences), compute gradients and update weights. Repeat on a new batch of sentences.